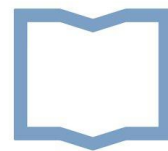
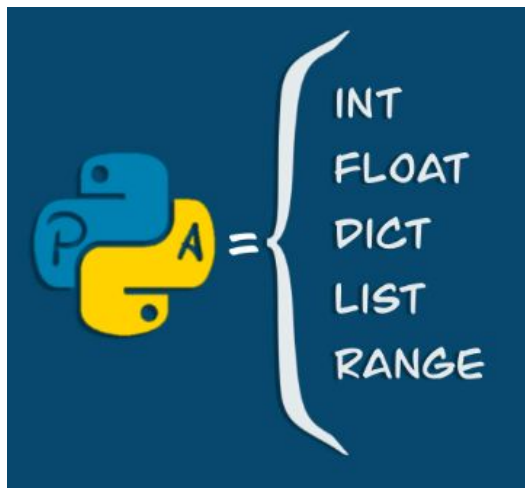


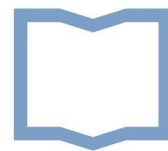
Laboratorio de programación II



Universidad
Nacional
de San Martín

Unidad 13 - Elementos y Variables



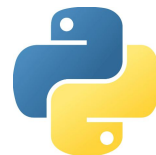
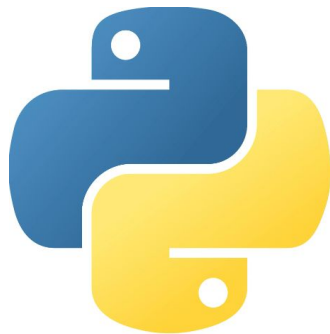


Universidad
Nacional
de San Martín

Python

¿Qué es ?

Python es un lenguaje de programación de alto nivel que se destaca por su sintaxis clara y legible, lo que lo hace muy popular tanto para principiantes como para programadores experimentados. Los programas en Python suelen organizarse en archivos con extensión ".py" y se ejecutan utilizando un intérprete de Python.



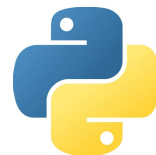


Universidad
Nacional
de San Martín

Python

Elementos que lo componen

- Palabras reservadas (keywords)
- Funciones integradas (built-in functions)
- Literales
- Operadores
- Delimitadores





Python

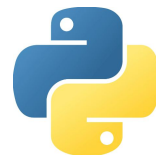
Estructura de un código

```
base= 5  
altura= 3  
superficie=base*altura  
print(superficie)
```

Se recomienda que cada línea contenga una única instrucción

```
base= 5  
altura= 3  
superficie=base* \  
altura  
print(superficie)
```

Si una instrucción es muy larga, se puede dividir en varias líneas usando el carácter contrabarra (\)





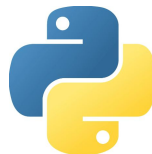
Python

Palabras reservadas

Las palabras reservadas de Python son las que forman el núcleo del lenguaje Python

False - await - else - import - pass - None - break - except - in - raise
- True - class - finally - is - return - and - continue - for - lambda - try
- as - def - from - nonlocal - while - assert - del - global - not - with -
async - elif - if - or - yield

Estas palabras no pueden utilizarse para nombrar otros elementos, pero pueden aparecer en cadenas de texto.



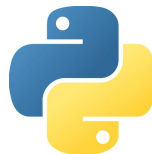


Python

Literales

Los literales son valores constantes que pueden ser utilizados directamente en el código Python sin necesidad de ser calculados o evaluados.

1. Literales numéricos: Representan valores numéricos, como enteros o números de punto flotante. Por ejemplo:
 - 42 (entero)
 - 3.14 (número de punto flotante)
2. Literales de cadena de texto: Representan secuencias de caracteres. Pueden estar encerrados entre comillas simples (' '), comillas dobles (" "), o comillas triples (''' ' o "" "" '''). Por ejemplo:
 - 'Hola mundo'
 - "Python es genial"
 - '''Este es un literal de cadena multilinea'''
3. Literales booleanos: Representan los valores verdadero (True) y falso (False). Por ejemplo:
 - True
 - False
4. Literales de secuencia: Representan listas, tuplas y diccionarios. Por ejemplo:
 - [1, 2, 3] (lista)
 - (1, 2, 3) (tupla)
 - { 'a': 1, 'b': 2, 'c': 3 } (diccionario)



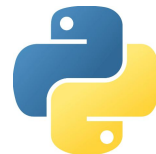
Python

Operadores Aritméticos

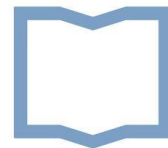
Los operadores aritméticos toman dos operandos como entrada, realizan un cálculo y devuelven el resultado

OPERADOR	DESCRIPCIÓN	USO
+	Suma entre los operandos	$12 + 3 = 15$
-	Resta entre los operandos	$12 - 3 = 9$
*	Multiplicación entre los operandos	$12 * 3 = 36$
/	División entre los operandos	$12 / 3 = 4$
%	Resto de la división entre los operandos	$16 \% 3 = 1$
**	Potencia de los operandos	$12 ** 3 = 1728$
//	División con resultado de número entero	$18 // 5 = 3$

Para obtener el resultado en tipo flotante, uno de los operandos también debe ser de tipo flotante.



Python

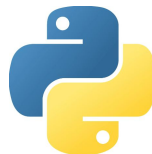


Universidad
Nacional
de San Martín

Operadores Relacionales

Un operador relacional se emplea para comparar y establecer la relación entre ellos. Devuelve un valor booleano (true o false) basado en la condición.

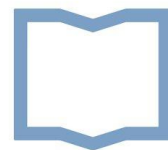
OPERADOR	DESCRIPCIÓN	USO
>	Devuelve True si el operador de la izquierda es mayor que el operador de la derecha	12 > 3 devuelve True
<	Devuelve True si el operador de la derecha es mayor que el operador de la izquierda	12 < 3 devuelve False
==	Devuelve True si ambos operandos son iguales	12 == 3 devuelve False
>=	Devuelve True si el operador de la izquierda es mayor o igual que el operador de la derecha	12 >= 3 devuelve True
<=	Devuelve True si el operador de la derecha es mayor o igual que el operador de la izquierda	12 <= 3 devuelve False
!=	Devuelve True si ambos operandos no son iguales	12 != 3 devuelve True



Python

Operadores Bit a Bit

Un operador bit a bit realiza operaciones en los operandos bit a bit.

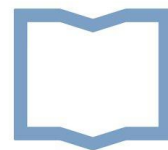


Universidad
Nacional
de San Martín

OPERADOR	DESCRIPCIÓN	USO
&	Realiza bit a bit la operación AND en los operandos	$a \& b = 2$ (Binario: $10 \& 11 = 10$)
	Realiza bit a bit la operación OR en los operandos	$a b = 3$ (Binario: $10 11 = 11$)
^	Realiza bit a bit la operación XOR en los operandos	$a \wedge b = 1$ (Binario: $10 \wedge 11 = 01$)
~	Realiza bit a bit la operación NOT bit a bit. Invierte cada bit en el operando	$\sim a = -3$ (Binario: $\sim(00000010) = (11111101)$)
>>	Realiza un desplazamiento a la derecha bit a bit. Desplaza los bits del operador de la izquierda a la derecha tantos bits como indica el operador de la derecha	$a \gg b = 0$ (Binario: $00000010 \gg 00000011 = 0$)
<<	Realiza un desplazamiento a la izquierda bit a bit. Desplaza los bits del operando de la izquierda a la izquierda tantos bits como especifique el operador de la derecha	$a \ll b = 16$ (Binario: $00000010 \ll 00000011 = 00001000$)



Python

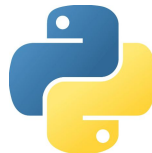


Universidad
Nacional
de San Martín

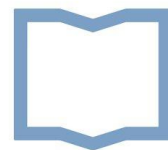
Operadores de Asignación

Se utiliza un operador de asignación para asignar valores a una variable. Esto generalmente se combina con otros operadores (como aritmética, bit a bit) donde la operación se realiza en los operandos y el resultado se asigna al operando izquierdo.

OPERADOR	DESCRIPCIÓN
=	a = 5. El valor 5 es asignado a la variable a
+=	a += 5 es equivalente a a = a + 5
-=	a -= 5 es equivalente a a = a - 5
*=	a *= 3 es equivalente a a = a * 3
/=	a /= 3 es equivalente a a = a / 3
%=	a %= 3 es equivalente a a = a % 3
**=	a **= 3 es equivalente a a = a * 3
//=	a //= 3 es equivalente a a = a // 3
&=	a &= 3 es equivalente a a = a & 3
=	a = 3 es equivalente a a = a 3
^=	a ^= 3 es equivalente a a = a ^ 3
>>=	a >>= 3 es equivalente a a = a >> 3
<<=	a <<= 3 es equivalente a a = a << 3



Python

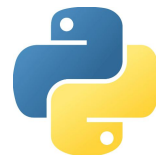


Universidad
Nacional
de San Martín

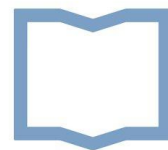
Operadores Lógicos

Se utiliza un operador lógico para tomar una decisión basada en múltiples condiciones. Los operadores lógicos utilizados en Python son and, or y not.

OPERADOR	DESCRIPCIÓN	USO
and	Devuelve True si ambos operandos son True	a and b
or	Devuelve True si alguno de los operandos es True	a or b
not	Devuelve True si alguno de los operandos es False	not a



Python



Universidad
Nacional
de San Martín

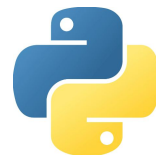
Operador de pertenencia

Un operador de pertenencia se emplea para identificar pertenencia en alguna secuencia (listas, strings, tuplas).

in y **not in** son operadores de pertenencia.

in devuelve **True** si el valor especificado se encuentra en la secuencia. En caso contrario devuelve **False**.

not in devuelve **True** si el valor especificado no se encuentra en la secuencia. En caso contrario devuelve **False**.

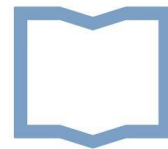


Python

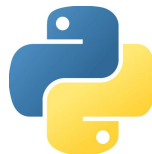
Delimitadores

Los delimitadores son los caracteres que permiten delimitar, separar o representar expresiones.

'	"	#	\			
(	)	[	]	{	}	
,	:	.	;	@	=	->
+=	-=	*=	/=	//=	%=	@=
&=	=	^=	>>=	<<=	**=	



Universidad
Nacional
de San Martín





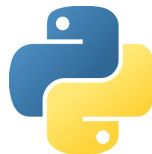
Python

Funciones integradas

Una función es un bloque de instrucciones agrupadas, que permiten reutilizar partes de un programa.

abs() - dict() - help() - min() - setattr() - all() - dir() - hex() - next() - slice() - any() - divmod() - id() - object() - sorted() - ascii() - enumerate() - input() - oct() - staticmethod() - bin() - eval() - int() - open() - str() - bool() - exec() - isinstance() - ord() - sum() - bytearray() - filter() - issubclass() - pow() - super() - bytes() - float() - iter() - print() - tuple() - callable() - format() - len() - property() - type() - chr() - frozenset() - list() - range() - vars() - classmethod() - getattr() - locals() - repr() - zip() - compile() - globals() - map() - reversed() - import() - complex() - hasattr() - max() - round() - delattr() - hash() - memoryview() - set()

Los nombres de las funciones integradas se pueden utilizar para nombrar variables, pero entonces las funciones ya no estarán disponibles en el programa. Si se eliminan las variables, las funciones vuelven a estar disponibles.





Python

Función print()

La función print() admite varios argumentos seguidos.

En el programa, los argumentos deben separarse por comas.

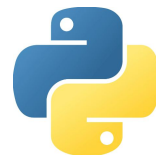
Los argumentos se muestran en el mismo orden y en la misma línea, separados por espacios.

```
print("Hola", "Mundo")
```

Al final de cada print(), Python añade automáticamente un salto de línea

```
print("Hola")  
print("Mundo")
```

Para generar una línea en blanco, se puede escribir una orden print() sin argumentos





Python

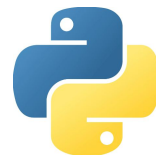
Función print()

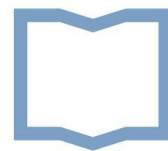
Como las comillas indican el principio y el final de una cadena, si se escribe el mismo formato de comillas dentro de comillas se produce un error de sintaxis.

```
print("Le pregunté "¿Cómo va?")
```

Para solucionar esto ponemos barra invertida

```
print("Le pregunté \"¿Cómo va?\")
```





Python

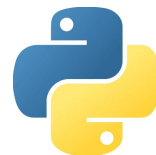
Función print()

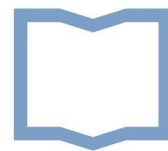
También podemos utilizar variables

```
nombre = "Maxi"  
print("¡Hola,", nombre, "!")
```

También podemos utilizar utilizar las cadenas "f"

```
nombre = "Juan"  
print(f"¡Hola, {nombre}!")
```





Python

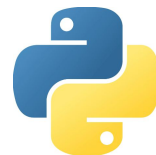
Función input()

La función input() permite obtener texto escrito por teclado

```
print("¿Cómo se llama?")
nombre = input()
print(f"Un gusto, {nombre}")
```

Podemos aprovechar que a la función input() se le puede enviar un argumento que se escribe en la pantalla

```
nombre = input("¿Cómo se llama? ")
print(f"Me alegro de conocerle, {nombre}")
```





Python

Conversión de tipos

De forma predeterminada, la función `input()` convierte la entrada en una cadena, aunque escribamos un número.

```
cantidad = input("Cuántos pesos tiene?: ")  
print(f"{cantidad} pesos son {round(cantidad / 1000.0, 2)} dólares")
```

La anterior línea daría un error, para solucionar esto podemos convertir a entero

```
cantidad = int(input(" Cuántos pesos tiene?: "))  
print(f"{cantidad} pesos son {round(cantidad / 1000.0, 2)} dólares")
```

e la misma manera, para que Python interprete la entrada como un número decimal, se debe utilizar la función `float()`





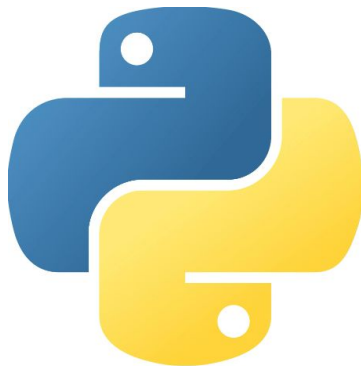
Python

Ejercicio integradores

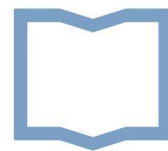
- Se desea calcular el área de un círculo dado su radio (float). Recuerda que el área de un círculo se calcula con la fórmula: $A = \pi * r^2$.
- Se necesita calcular el salario semanal de un trabajador en base a su salario por hora (float) y el número de horas trabajadas a la semana (int).
- Desarrolla un programa que convierta una temperatura dada en grados Celsius (float) a grados Fahrenheit, utilizando la fórmula: $F = (C * 9/5) + 32$.
- Construye un algoritmo que calcule el perímetro de un rectángulo a partir de su longitud (float) y anchura (float), utilizando la fórmula: $P = 2 * (longitud + anchura)$.



Próxima unidad: Variables y tipos.



Laboratorio de programación II



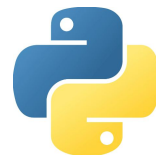
Universidad
Nacional
de San Martín

Unidad 14 - Variables, listas y condicionales.

Assignment

age = 25

Variable   Value





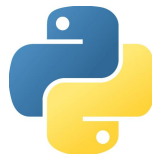
Variables

¿Qué son ?

Las variables en Python son como contenedores donde puedes almacenar datos

```
nombre = "Juan"  
edad = 25  
altura = 1.75  
fecha_de_nacimiento="25/05/2000"
```

Una cosa interesante sobre Python es que no necesitas especificar el tipo de datos que contendrá la variable.

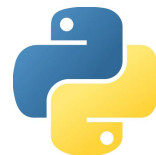


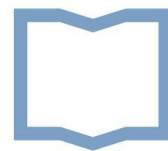


Variables

Python también tiene la función `isinstance()` que puedes usar para verificar si una variable es de un tipo específico.

```
variable = 10  
print(isinstance(variable, int)) # Esto imprimirá True si la variable es  
de tipo entero (int), de lo contrario, imprimirá False.
```



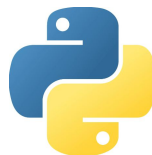


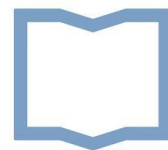
Listas

¿Qué son ?

Las listas son estructuras de datos muy flexibles, son conjuntos de elementos (números, cadenas, listas, etc), se delimitan por corchetes ([]) y los elementos se separan por comas.

```
numeros = [1, 2, 3, 4, 5]
```





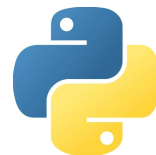
Listas

También pueden crear una lista vacía y luego agregar elementos a ella

```
frutas = []  
frutas.append("manzana")  
frutas.append("banana")  
frutas.append("cereza")
```

Pueden acceder a elementos individuales de la lista utilizando su índice, que comienza desde 0.

```
frutas = ["manzana", "banana", "cereza"]  
print(frutas[0]) # Imprime: manzana  
print(frutas[1]) # Imprime: banana
```





Listas

También pueden acceder a elementos desde el final de la lista utilizando índices negativos

```
frutas = ["manzana", "banana", "cereza"]  
print(frutas[-1]) # Imprime: cereza
```



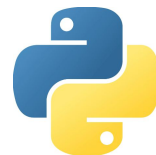


Listas

Slicing (rebanado) de listas

Pueden obtener una porción (slice) de la lista utilizando la sintaxis [inicio:fin]

```
numeros = [1, 2, 3, 4, 5]  
print(numeros[1:3]) # Imprime: [2, 3]
```



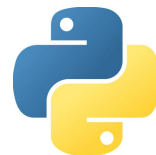


Listas

Modificación de elementos de la lista

Pueden cambiar el valor de un elemento de la lista asignándole un nuevo valor

```
frutas = ["manzana", "banana", "cereza"]  
frutas[1] = "kiwi"  
print(frutas) # Imprime: ["manzana", "kiwi", "cereza"]
```



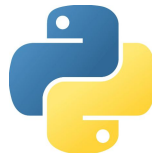


Listas

Borrado de elementos de la lista

Pueden eliminar elementos de la lista utilizando la palabra clave `del` o el método `remove()`. Por ejemplo:

```
frutas = ["manzana", "banana", "cereza"]  
del frutas[0] # Elimina la primera fruta  
print(frutas) # Imprime: ["banana", "cereza"]  
  
frutas.remove("banana") # Elimina la fruta "banana"  
print(frutas) # Imprime: ["cereza"]
```





Listas

Operaciones comunes con listas

- Para agregar elementos al final de la lista, utiliza el método `append()`.
- Para extender una lista con los elementos de otra lista, utiliza el método `extend()` o el operador `+`.
- Para encontrar la longitud de la lista, utiliza la función `len()`.
- Para buscar un elemento en la lista, utiliza el operador `in`.



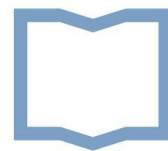


Listas

Operaciones comunes con listas

```
frutas = ["manzana", "banana"]  
frutas.append("cereza")  
print(frutas) # Imprime: ["manzana", "banana", "cereza"]  
  
otras_frutas = ["naranja", "uva"]  
frutas.extend(otras_frutas)  
print(frutas) # Imprime: ["manzana", "banana", "cereza", "naranja",  
"uva"]  
  
print(len(frutas)) # Imprime: 5  
  
print("manzana" in frutas) # Imprime: True
```





Listas

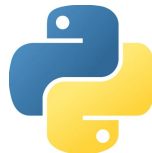
Ordenamiento de listas

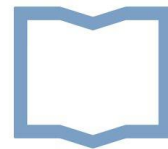
Pueden ordenar una lista utilizando el método `sort()`.

```
numeros = [3, 1, 4, 1, 5, 9, 2]
numeros.sort()
print(numeros) # Imprime: [1, 1, 2, 3, 4, 5, 9]
```

Pueden ordenar una lista utilizando el método `sort()`.

```
numeros.sort(reverse=True)
print(numeros) # Imprime: [9, 5, 4, 3, 2, 1, 1]
```



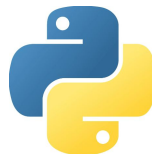


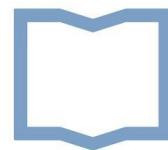
Listas

Obtener el índice de un elemento

Pueden obtener el índice de un elemento en la lista utilizando el método `index()`. Por ejemplo:

```
frutas = ["manzana", "banana", "cereza"]  
indice = frutas.index("banana")  
print(indice) # Imprime: 1
```



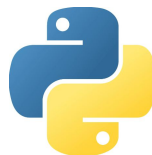


Listas

Contar ocurrencias de un elemento

Pueden contar cuántas veces aparece un elemento en la lista utilizando el método `count()`.

```
numeros = [1, 2, 3, 4, 1, 2, 3, 1]  
conteo = numeros.count(1)  
print(conteo) # Imprime: 3
```



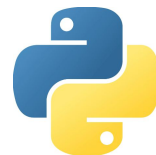


Listas

Reversión de una lista

Pueden revertir el orden de una lista utilizando el método `reverse()`. Por ejemplo:

```
frutas = ["manzana", "banana", "cereza"]  
frutas.reverse()  
print(frutas) # Imprime: ["cereza", "banana", "manzana"]
```





Listas

Eliminación de todos los elementos de la lista:

Pueden eliminar todos los elementos de una lista utilizando el método `clear()`.

```
frutas = ["manzana", "banana", "cereza"]  
frutas.clear()  
print(frutas) # Imprime: []
```





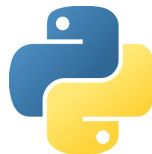
Condicionales

¿Qué son ?

Los condicionales en Python son herramientas que te permiten controlar el flujo de ejecución de tu programa. Son bifurcaciones en el desarrollo del programa:

```
edad = 20
```

```
if edad >= 18:  
    print("Eres mayor de edad.")
```

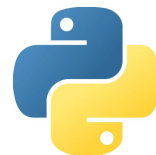




Condicionales

A menudo, también queremos ejecutar un bloque de código si la condición no es verdadera. Para eso, usamos el "else".

```
if edad >= 18:  
    print("Eres mayor de edad.")  
else:  
    print("Eres menor de edad.")
```



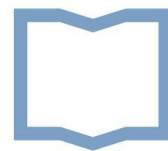


Condicionales

A veces, también queremos verificar múltiples condiciones. Para eso, usamos el "elif" (abreviatura de "else if").

```
if edad < 18:  
    print("Eres menor de edad.")  
elif edad == 18:  
    print("Tienes exactamente 18 años.")  
else:  
    print("Eres mayor de edad.")
```



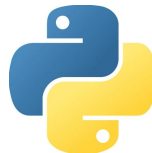


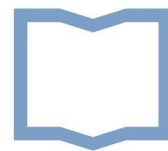
Condicionales

Finalmente, Python también nos permite combinar condiciones utilizando los operadores lógicos "and", "or" y "not". Por ejemplo:

```
temperatura = 25  
llueve = True
```

```
if temperatura > 20 and not llueve:  
    print("Es un buen día para salir.")  
elif temperatura <= 20 or llueve:  
    print("Mejor quedarse en casa.")
```





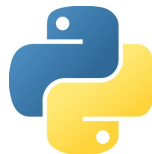
Condicionales

Cuestiones a tener en cuenta:

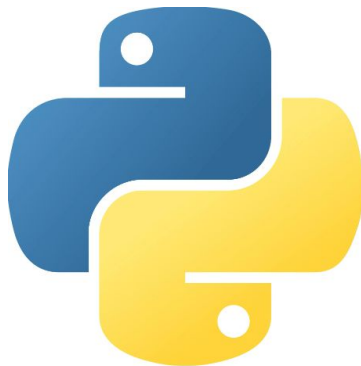
Lo que NO se permite es que en un mismo bloque haya instrucciones con distintos indentados. Dependiendo del orden de los indentado, el mensaje de error al intentar ejecutar el programa será diferente.

Si necesitamos no hacer nada dentro de un if podemos utilizar la palabra “pass”

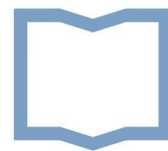
```
llueve = True  
if temperatura > 20:  
    pass
```



Próxima unidad: Ciclos y objetos.



Laboratorio de computación II

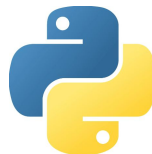


Universidad
Nacional
de San Martín

Unidad 15 - Ciclos y range.



```
for i in range(n):  
    # Código
```

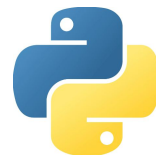




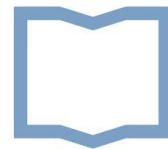
Ciclo For

El ciclo for en Python es una estructura de control que nos permite iterar sobre una secuencia (como una lista, una tupla, un diccionario, un conjunto o una cadena de caracteres)

```
for variable in secuencia:  
    # cuerpo del ciclo
```



Ejemplos



Universidad
Nacional
de San Martín

Supongamos que tenemos una lista de números y queremos imprimir cada número

```
numeros = [1, 2, 3, 4, 5]  
  
for numero in numeros:  
    print(numero)
```



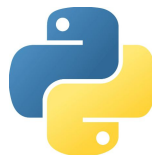


Ejemplos

También podemos usar for para iterar sobre una cadena de caracteres

```
cadena = "Python"  
  
for caracter in cadena:  
    print(caracter)
```

El ciclo for en Python utilizarse con cualquier objeto iterable.





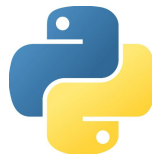
Ejemplos

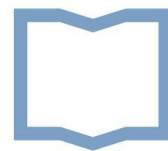
Imaginemos que queremos calcular la suma de todos los números en una lista. Usaremos un acumulador para almacenar la suma a medida que iteramos sobre la lista.

```
numeros = [1, 2, 3, 4, 5]
suma = 0 # Inicializamos el acumulador en 0

for numero in numeros:
    suma += numero # Añadimos cada número al
    acumulador

print("La suma de los números es:", suma)
```





Ejemplos

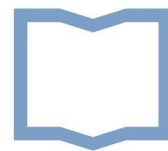
Podemos usar un acumulador para contar cuántos elementos en una lista cumplen con una condición específica.

```
numeros = [1, 2, 3, 4, 5]
contador = 0 # Inicializamos el contador en 0

for numero in numeros:
    if numero > 3:
        contador += 1 # Incrementamos el contador cuando la
        condición se cumple

print("Hay", contador, "números mayores que 3.")
```

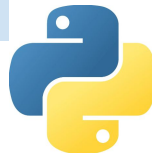




Ejemplos

Supongamos que queremos concatenar todas las palabras en una lista en una sola cadena, separadas por espacios.

```
palabras = ["Hola", "mundo", "desde", "Python"]  
cadena_concatenada = "" # Inicializamos la cadena vacía  
  
for palabra in palabras:  
    cadena_concatenada += palabra + " " # Añadimos cada palabra seguida  
    de un espacio  
  
print("La cadena concatenada es:", cadena_concatenada)
```





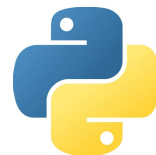
Ejemplos

Otro ejemplo de acumulador es calcular el producto de todos los números en una lista.

```
numeros = [1, 2, 3, 4, 5]
producto = 1 # Inicializamos el acumulador en 1

for numero in numeros:
    producto *= numero # Multiplicamos cada número al acumulador

print("El producto de los números es:", producto)
```





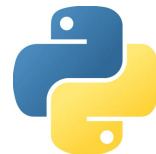
Ejemplos

Imaginemos que queremos encontrar el número más grande en una lista de números. Usaremos una variable testigo para almacenar el mayor número encontrado hasta el momento.

```
numeros = [3, 7, 2, 8, 4, 10, 1]
maximo = numeros[0] # Inicializamos la variable testigo con el primer elemento de la lista

for numero in numeros:
    if numero > maximo:
        maximo = numero # Actualizamos la variable testigo si encontramos un número mayor

print("El número más grande es:", maximo)
```





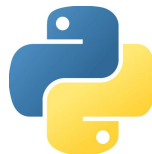
Ejemplos

Supongamos que queremos verificar si hay algún número par en una lista de números. Usaremos una bandera para indicar si encontramos un número par.

```
numeros = [3, 7, 2, 8, 4, 10, 1]
hay_par = False # Inicializamos la bandera en False

for numero in numeros:
    if numero % 2 == 0:
        hay_par = True # Cambiamos la bandera a True si encontramos un número
        par
        break # Podemos salir del ciclo una vez que encontremos un número par

if hay_par:
    print("Hay al menos un número par en la lista.")
else:
    print("No hay números pares en la lista.")
```

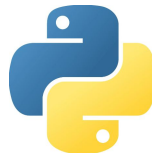




Ejemplos

Imaginemos que tenemos una matriz (lista de listas) y queremos sumar todos los elementos de la matriz.

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
suma_total = 0 # Inicializamos el acumulador en 0  
  
for fila in matriz: # Primer ciclo recorre cada fila de la matriz  
    for elemento in fila: # Segundo ciclo recorre cada elemento en la fila  
        suma_total += elemento # Sumamos el elemento al acumulador  
  
print("La suma de todos los elementos en la matriz es:", suma_total)
```

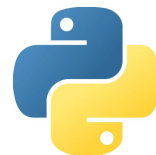




Ciclo while

Los ciclos while en Python son una estructura de control que permite ejecutar un bloque de código repetidamente mientras una condición específica sea verdadera.

```
while condición:  
    # cuerpo del ciclo
```



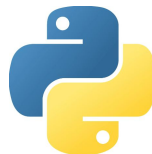


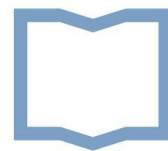
Ejemplos

En este ejemplo, el ciclo while seguirá ejecutándose mientras contador sea menor que 5. En cada iteración, se imprime el valor de contador y luego se incrementa en 1. Cuando contador llega a 5, la condición `contador < 5` se vuelve falsa y el ciclo termina.

```
contador = 0
```

```
while contador < 5:  
    print("Contador:", contador)  
    contador += 1
```



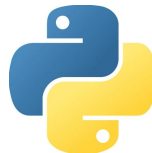


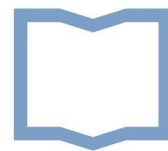
Ejemplos

En este caso, el ciclo while continuará hasta que el usuario escriba "salir". La función input solicita al usuario que ingrese un texto, y el ciclo se repite hasta que la entrada del usuario, convertida a minúsculas, sea igual a "salir".

```
# Ejemplo de un ciclo while con una condición de salida específica  
respuesta = ""
```

```
while respuesta.lower() != "salir":  
    respuesta = input("Escribe 'salir' para terminar el programa: ")  
    print("Ingresaste:", respuesta)
```





Casos especiales

Break termina el ciclo inmediatamente, independientemente de la condición.

```
contador = 0

while True:
    print("Contador:", contador)
    contador += 1
    if contador >= 5:
        break # Sale del ciclo cuando contador es mayor o igual a 5
```



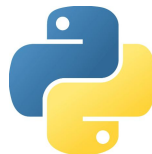


Casos especiales

Continue salta el resto del bloque de código en la iteración actual y pasa a la siguiente iteración del ciclo.

```
contador = 0

while contador < 5:
    contador += 1
    if contador == 3:
        continue # Salta el resto del bloque cuando contador es 3
    print("Contador:", contador)
```





Casos especiales

Python permite usar una cláusula else con un ciclo while. El bloque else se ejecuta cuando la condición del while se vuelve falsa, pero no si el ciclo se interrumpe con un break.

```
contador = 0

while contador < 5:
    print("Contador:", contador)
    contador += 1
else:
    print("El ciclo ha terminado")
```





Range

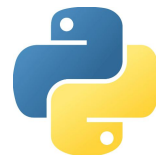
¿Qué es ?

La función range en Python se utiliza para generar una secuencia de números. Es comúnmente utilizada en ciclos for para iterar un número específico de veces.

`range(inicio, fin, paso)`

- **inicio** es el número inicial de la secuencia (inclusive).
- **fin** es el número en el que la secuencia se detiene (exclusivo).
- **paso** es la diferencia entre cada par de números consecutivos en la secuencia.

Range solo admite pasos de números enteros!





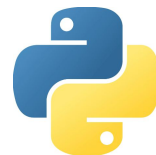
Range

Cuando se pasa un solo argumento, range genera números desde 0 hasta fin-1.

```
for i in range(5):  
    print(i)  
# en este caso, la secuencia generada es [0, 1, 2, 3, 4].
```

Cuando se pasan dos argumentos, range genera números desde inicio hasta fin-1.

```
for i in range(2, 5):  
    print(i)
```



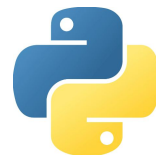


Range

Cuando se pasan tres argumentos, range genera números desde inicio hasta fin-1, incrementando por paso.

```
for i in range(1, 10, 2):  
    print(i)
```

En este caso, la secuencia generada es [1, 3, 5, 7, 9].



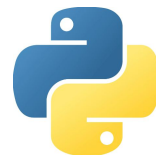


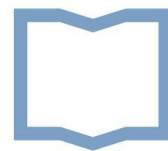
Range

También puede trabajar con valores negativos para inicio, fin y paso.

```
for i in range(10, 0, -2):  
    print(i)
```

En este caso, la secuencia generada es [10, 8, 6, 4, 2].

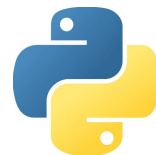


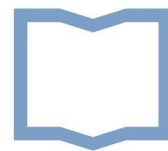


Range

Pueden convertir un objeto range en una lista usando la función list.

```
lista = list(range(5))  
print(lista) # obtenemos [0, 1, 2, 3, 4]
```

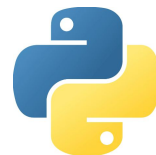




Range

Iterar sobre índices de una lista

```
mi_lista = ['a', 'b', 'c', 'd']  
  
for i in range(len(mi_lista)):  
    print(f"Índice {i} tiene el valor {mi_lista[i]}")
```

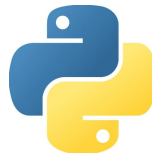




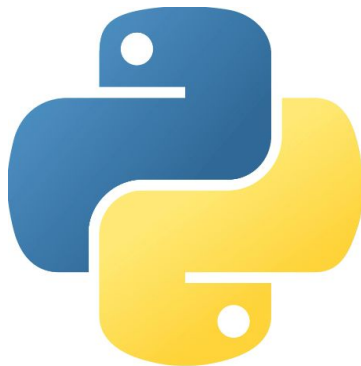
Ejercicio integrador

Crear un programa que cumpla con los siguientes requisitos:

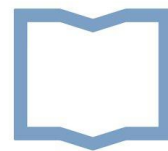
- **Entrada de Datos:**
Crea una lista llamada temperaturas que contenga las temperaturas diarias (en grados Celsius) de un mes (30 días). Los datos serán ingresados manualmente por el usuario.
- **Cálculo de Promedios:**
Utilizando un bucle for, calcula la temperatura media del mes y almacénala en una variable llamada promedio_mensual.
- **Detección de Días Fríos y Cálidos:**
Utilizando un bucle while, recorre la lista temperaturas y cuenta cuántos días tuvieron una temperatura por debajo de 20 grados (días fríos) y cuántos días tuvieron una temperatura por encima de 30 grados (días cálidos). Almacena estos contadores en variables llamadas dias_frios y dias_calidos respectivamente.
- **Generación de un Informe Semanal:**
Divide los datos en semanas (cada semana tiene 7 días) y calcula la temperatura media de cada semana utilizando un bucle for y la función range. Almacena estos promedios semanales en una lista llamada promedios_semanales.
- **Visualización de Resultados:**
Imprime el promedio_mensual, los valores de dias_frios y dias_calidos, y los valores de promedios_semanales.



Próxima unidad: Diccionarios, Tuplas y Funciones.



Laboratorio de computación II



Universidad
Nacional
de San Martín

Unidad 16 - Conjuntos y objetos .

```

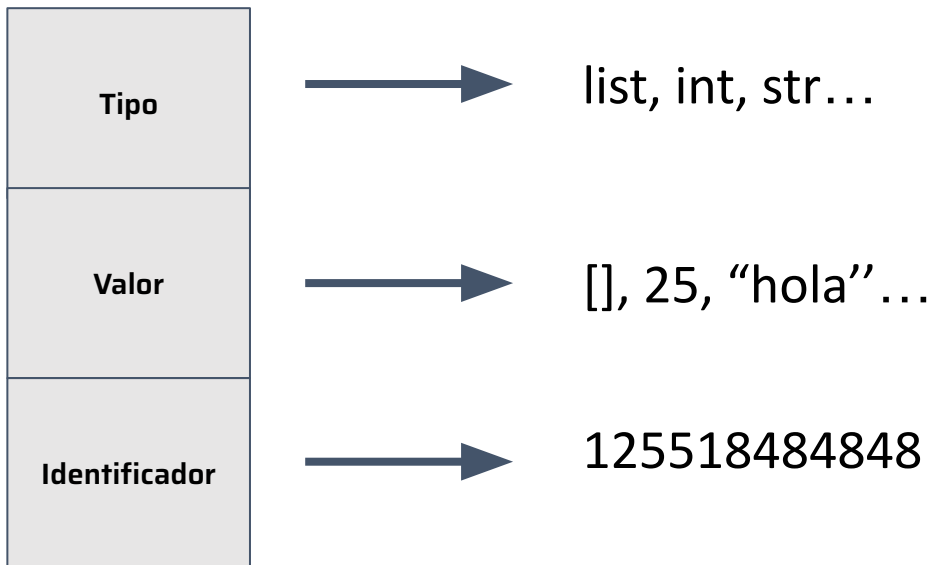
>>> var_x = 10
>>> var_x
10
>>> type(var_x)
<class 'int'>
>>> id(var_x)
140313105668688

```





Estructura de un objeto





Objetos

Objetos Mutables

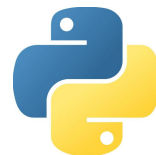
Los objetos mutables son aquellos que pueden ser modificados después de ser creados. Puedes cambiar su contenido sin necesidad de crear un nuevo objeto.

- Listas (list)
- Diccionarios (dict)
- Conjuntos (set)

Objetos Inmutables

Los objetos inmutables son aquellos que, una vez creados, no pueden ser modificados. Si intentas cambiar el valor de un objeto inmutable, en realidad estarás creando un nuevo objeto.

- Números (enteros, flotantes)
- Cadenas de texto (str)
- Tuplas (tuple)



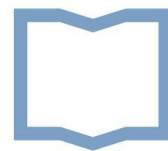


Importancia de la Inmutabilidad

La inmutabilidad es importante porque:

- Seguridad y Consistencia: Los objetos inmutables no pueden ser modificados accidentalmente después de su creación.
- Hashing: Solo los objetos inmutables pueden ser usados como claves en un diccionario o elementos de un conjunto.
- Facilita el Depurado: El comportamiento de los objetos inmutables es más predecible y fácil de seguir en el código.



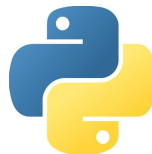


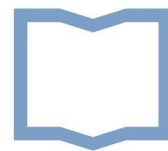
Ejemplos de mutabilidad

Si modificamos una lista y imprimimos su identificador

```
my_list = [1, 2, 3]
print(my_list)      # Imprime: [1, 2, 3]
print(id(my_list))  # Imprime una dirección de memoria

my_list[0] = 10
print(my_list)      # Imprime: [10, 2, 3]
print(id(my_list))  # Imprime la misma dirección de memoria
```

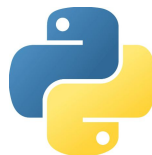




Ejemplos de mutabilidad

En cambio en una cadena de caracteres

```
my_str = "hello"  
print(my_str)      # Imprime: hello  
print(id(my_str))  # Imprime una dirección de memoria  
  
my_str = "world"  
print(my_str)      # Imprime: world  
print(id(my_str))  # Imprime una dirección de memoria diferente
```





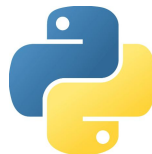
Problema del objeto mutable

Acá `lista_original` y `lista_nueva` apuntan al mismo objeto en memoria. Por lo tanto, cuando modificamos `lista_nueva`, también estamos modificando `lista_original`.

```
lista_original = [1, 2, 3]
lista_nueva = lista_original # Asignamos lista_original a lista_nueva

# Modificamos lista_nueva
lista_nueva.append(4)

# Observamos ambas listas
print("lista_original:", lista_original) # Output: [1, 2, 3, 4]
print("lista_nueva:", lista_nueva)      # Output: [1, 2, 3, 4]
```





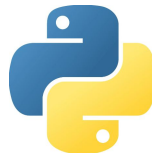
Solución del problema

Para solucionar el problema de la mutabilidad deberíamos realizar una copia con el método `copy()`

```
lista_original = [1, 2, 3]
lista_nueva = lista_original.copy() # Creamos una copia de la lista original

# Modificamos lista_nueva
lista_nueva.append(4)

# Observamos ambas listas
print("lista_original:", lista_original) # Output: [1, 2, 3]
print("lista_nueva:", lista_nueva)      # Output: [1, 2, 3, 4]
```





Conjuntos

Un conjunto es una colección de elementos únicos y no ordenados. A diferencia de las listas y las tuplas, los conjuntos no permiten elementos duplicados.

```
conjunto_vacio = set()  
print(conjunto_vacio) # Output: set()
```



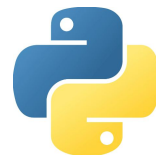


Conjuntos con Elementos

Podemos inicializar conjuntos de la siguiente manera

```
conjunto = {1, 2, 3, 4}  
print(conjunto) # Output: {1, 2, 3, 4}
```

```
conjunto_mixto = {1, "hola", 3.14, (1, 2)}  
print(conjunto_mixto) # {1, "hola", 3.14, (1, 2)}
```





Conjuntos con Elementos

También podemos crear conjuntos a partir de otras colecciones como listas o cadenas

```
lista = [1, 2, 3, 4, 4, 5]
conjunto_desde_lista = set(lista)
print(conjunto_desde_lista) # Output: {1, 2, 3, 4, 5}

cadena = "hola"
conjunto_desde_cadena = set(cadena)
print(conjunto_desde_cadena) # Output: {'h', 'o', 'l', 'a'}
```



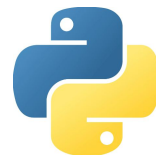


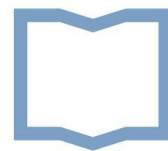
Operaciones Básicas con Conjuntos

Podemos añadir elementos a un conjunto utilizando el método `add()` y eliminarlos usando el método `remove()`

```
conjunto = {1, 2, 3}  
conjunto.add(4)  
print(conjunto) # Output: {1, 2, 3, 4}
```

```
conjunto.remove(2)  
print(conjunto) # Output: {1, 3, 4}
```

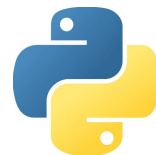


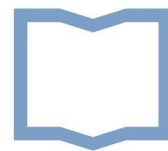


Operaciones Básicas con Conjuntos

Podemos verificar si un elemento está en un conjunto usando el operador in

```
conjunto = {1, 2, 3}  
print(2 in conjunto) # Output: True  
print(5 in conjunto) # Output: False
```





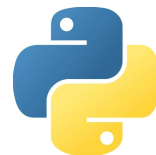
Operaciones Matemáticas con Conjuntos

La unión de dos conjuntos contiene todos los elementos de ambos conjuntos. Utilizamos el operador `|` o el método `union()`:

```
conjunto1 = {1, 2, 3}  
conjunto2 = {3, 4, 5}
```

```
union = conjunto1 | conjunto2  
print(union) # Output: {1, 2, 3, 4, 5}
```

```
union = conjunto1.union(conjunto2)  
print(union) # Output: {1, 2, 3, 4, 5}
```





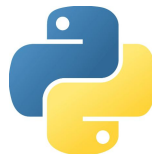
Operaciones Matemáticas con Conjuntos

La intersección de dos conjuntos contiene solo los elementos que están en ambos conjuntos. Utilizamos el operador `&` o el método `intersection()`:

```
conjunto1 = {1, 2, 3}  
conjunto2 = {3, 4, 5}
```

```
interseccion = conjunto1 & conjunto2  
print(interseccion) # Output: {3}
```

```
interseccion = conjunto1.intersection(conjunto2)  
print(interseccion) # Output: {3}
```





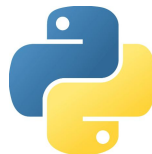
Operaciones Matemáticas con Conjuntos

La diferencia entre dos conjuntos contiene los elementos que están en el primer conjunto pero no en el segundo. Utilizamos el operador - o el método difference()

```
conjunto1 = {1, 2, 3}  
conjunto2 = {3, 4, 5}
```

```
diferencia = conjunto1 - conjunto2  
print(diferencia) # Output: {1, 2}
```

```
diferencia = conjunto1.difference(conjunto2)  
print(diferencia) # Output: {1, 2}
```





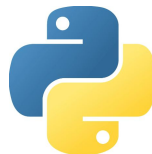
Operaciones Matemáticas con Conjuntos

La diferencia simétrica contiene los elementos que están en cualquiera de los conjuntos, pero no en ambos. Utilizamos el operador $\wedge$ o el método `symmetric_difference()`:

```
conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}
```

```
diferencia_simetrica = conjunto1 ^ conjunto2
print(diferencia_simetrica) # Output: {1, 2, 4, 5}
```

```
diferencia_simetrica = conjunto1.symmetric_difference(conjunto2)
print(diferencia_simetrica) # Output: {1, 2, 4, 5}
```





Operaciones Matemáticas con Conjuntos

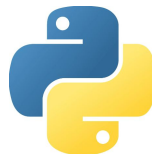
Podemos verificar si un conjunto es subconjunto de otro usando `issubset()` y si es un superconjunto usando `issuperset()`:

```
a = {1, 2, 3}
```

```
b = {1, 2, 3, 4, 5}
```

```
print(a.issubset(b)) # Output: True
```

```
print(b.issuperset(a)) # Output: True
```





Ejercicio integrador

Crear un programa que cumpla con los siguientes requisitos:

Implementa un script que realice operaciones con conjuntos para gestionar el inventario de una tienda. El script utilizará tres conjuntos `inventario_tienda1`, `inventario_tienda2` y `productos_defectuosos`, representando los productos disponibles en dos tiendas y los productos defectuosos en ambas tiendas, respectivamente.

Ejemplo:

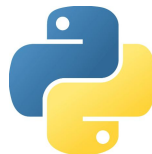
```
inventario_tienda1 = {"manzanas", "naranjas", "uvas", "platanos"}
```

```
inventario_tienda2 = {"peras", "uvas", "platanos", "kiwis"}
```

```
productos_defectuosos = {"uvas", "kiwis"}
```

El script debe realizar las siguientes operaciones:

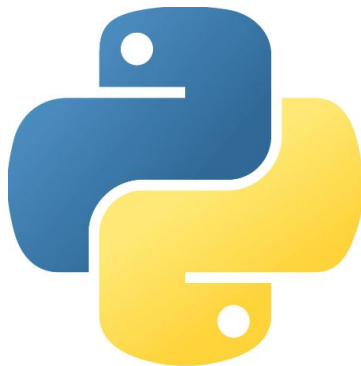
- Encontrar la lista completa de productos disponibles en ambas tiendas (unión de `inventario_tienda1` y `inventario_tienda2`).
- Encontrar los productos que están disponibles en ambas tiendas (intersección de `inventario_tienda1` y `inventario_tienda2`).
- Encontrar los productos que están disponibles en cualquiera de las tiendas pero no en ambas (diferencia simétrica entre `inventario_tienda1` y `inventario_tienda2`).
- Encontrar los productos disponibles en ambas tiendas que no son defectuosos.
- Verificar si todos los productos defectuosos están en el inventario de la tienda 1 (subconjunto).
- El script debe mostrar los resultados de cada operación en la consola.





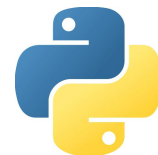
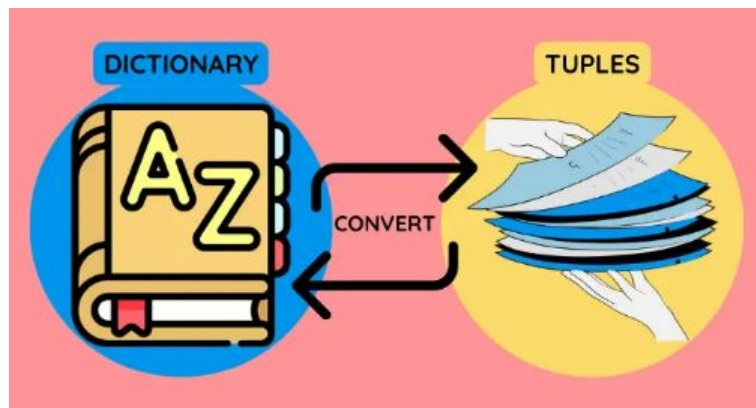
Universidad
Nacional
de San Martín

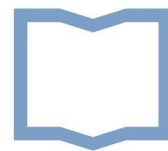
Próxima unidad: Diccionarios, Tuplas y Funciones.



Laboratorio de computación II

Unidad 17 - Diccionarios, Tuplas y Funciones .

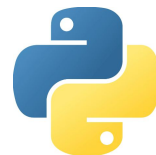




Diccionarios

Un diccionario es una colección desordenada, modificable e indexada de elementos. En lugar de usar índices numéricos como las listas, los diccionarios usan claves únicas para acceder a sus valores.

```
mi_diccionario = {  
    "clave1": "valor1",  
    "clave2": "valor2",  
    "clave3": "valor3"  
}
```





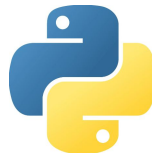
Diccionario con Elementos

Pueden crear un diccionario vacío y luego añadirle elementos, o puedes crear un diccionario con elementos directamente.

```
diccionario_vacio = {}
```

Diccionario con Elementos

```
mi_diccionario = {  
    "nombre": "Juan",  
    "edad": 25,  
    "ciudad": "Madrid"  
}
```





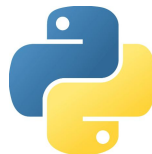
Acceder a los Elementos del Diccionario

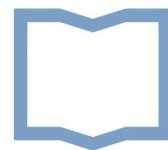
Para acceder a un valor en el diccionario, usas la clave correspondiente entre corchetes [].

```
print(mi_diccionario["nombre"]) # Output: Juan  
print(mi_diccionario["edad"])  # Output: 25
```

Modificar Elementos del Diccionario

```
mi_diccionario["edad"] = 26  
print(mi_diccionario["edad"]) # Output: 26
```

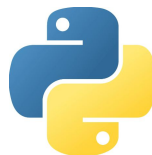




Añadir Nuevos Elementos al Diccionario

Para añadir un nuevo par clave-valor, simplemente asignas un valor a una nueva clave.

```
mi_diccionario["profesión"] = "Ingeniero"  
print(mi_diccionario)
```





Eliminar Elementos del Diccionario

Puedes eliminar elementos del diccionario usando el método `pop()` o la palabra clave `del`.

```
mi_diccionario.pop("ciudad")  
print(mi_diccionario)
```

```
del mi_diccionario["profesión"]  
print(mi_diccionario)
```

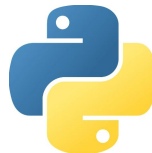




Métodos Útiles de Diccionarios

- `keys()`: Devuelve una vista de todas las claves en el diccionario.
- `values()`: Devuelve una vista de todos los valores en el diccionario.
- `items()`: Devuelve una vista de todos los pares clave-valor en el diccionario.

```
print(mi_diccionario.keys()) }  
# Output: dict_keys(['nombre', 'edad'])  
print(mi_diccionario.values())  
# Output: dict_values(['Juan', 26])  
print(mi_diccionario.items()) }  
# Output: dict_items([('nombre', 'Juan'), ('edad', 26)])
```



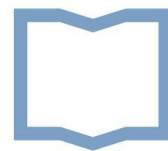


Iterar Sobre un Diccionario

Puedes iterar sobre un diccionario usando un bucle for. Puedes iterar sobre las claves, los valores o ambos.

```
for clave in mi_diccionario:  
    print(clave, mi_diccionario[clave])  
  
for clave, valor in mi_diccionario.items():  
    print(clave, valor)
```





Tuplas

Una tupla es una colección ordenada de elementos que puede contener diferentes tipos de datos (números, cadenas, listas, etc.). La principal característica de las tuplas es que son inmutables, es decir, una vez creadas, no se pueden modificar (añadir, eliminar o cambiar sus elementos).

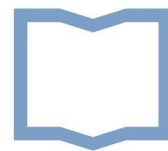
Sintaxis

Básica

```
mi_tupla = (1, 2, 3)
```

Las tuplas se definen con paréntesis () y los elementos se separan por comas.





Creación de una Tupla

Tupla Vacía

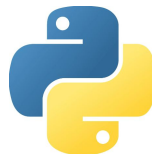
```
tupla_vacia = ()
```

Tupla con Elementos

```
mi_tupla = ("manzana", "banana", "cereza")
```

Las tuplas se definen con paréntesis () y los elementos se separan por comas.

```
mi_tupla = "manzana", "banana", "cereza"
```





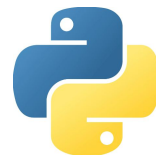
Creación de una Tupla

Para una tupla de un solo elemento, debes añadir una coma después del elemento

```
tupla_un_elemento = ("manzana",)
```

Acceder a los Elementos de una Tupla

```
print(mi_tupla[0]) # Output: manzana  
print(mi_tupla[1]) # Output: banana
```

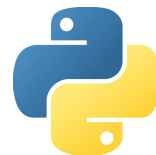




Desempaquetado de Tuplas

Pueden asignar los elementos de una tupla a variables individuales.

```
fruta1, fruta2, fruta3 = mi_tupla  
print(fruta1) # Output: manzana  
print(fruta2) # Output: banana  
print(fruta3) # Output: cereza
```



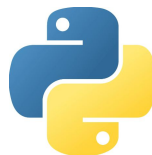


Métodos y Operaciones con Tuplas

Las tuplas tienen algunos métodos integrados y soportan varias operaciones:

- `count(x)`: Devuelve el número de veces que `x` aparece en la tupla.
- `index(x)`: Devuelve el índice de la primera aparición de `x` en la tupla.

```
mi_tupla = (1, 2, 2, 3, 4)
print(mi_tupla.count(2)) # Output: 2
print(mi_tupla.index(3)) # Output: 3
```

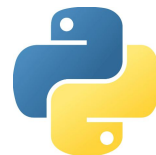




Inmutabilidad de las Tuplas

Una vez creada una tupla, no se pueden modificar sus elementos. Intentar hacerlo resultará en un error.

```
mi_tupla = (1, 2, 3)
# mi_tupla[1] = 4 # Esto dará un error TypeError:
'tuple' object does not support item assignment
```





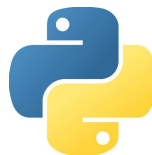
Usos Comunes de las Tuplas

Las tuplas se usan comúnmente cuando quieren almacenar una colección de elementos que no deben cambiar a lo largo del programa. Por ejemplo, puedes usar tuplas para:

Almacenar coordenadas (x, y).

Almacenar días de la semana.

Como claves en diccionarios (ya que las claves deben ser inmutables).





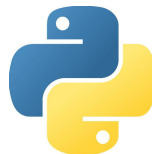
Funciones

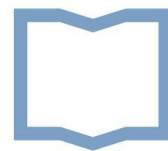
Una función es un bloque de código reutilizable que realiza una tarea específica. Puedes definir una función una vez y luego llamarla tantas veces como sea necesario, lo que ayuda a mantener tu código limpio y organizado.

Definición de una

Función

```
def nombre_de_la_funcion():  
    # Cuerpo de la función  
    print("Hola, soy una función")
```



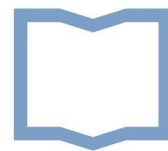


Llamar a una Función

Para ejecutar una función, simplemente llamas su nombre seguido de paréntesis ().

```
def nombre_de_la_funcion():  
    # Cuerpo de la función  
    print("Hola, soy una función")  
nombre_de_la_funcion() # Output: Hola, soy una función
```

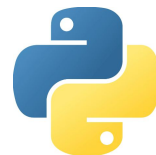




Funciones con Valores de Retorno

Las funciones pueden devolver valores usando la palabra clave return. Esto te permite capturar el resultado de la función y usarlo en otra parte de tu código.

```
def suma(a, b):  
    return a + b  
  
resultado = suma(3, 4)  
print(resultado) # Output: 7
```



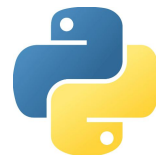


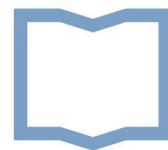
Funciones con Parámetros

Las funciones pueden tomar parámetros, que son variables que pasas a la función para que realice operaciones con ellas.

```
def saludar(nombre):  
    print(f"Hola, {nombre}")
```

```
saludar("Juan") # Output: Hola, Juan  
saludar("Ana") # Output: Hola, Ana
```



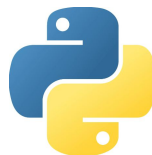


Funciones con Parámetros por Defecto

Pueden definir valores por defecto para los parámetros. Si no se proporciona un valor al llamar a la función, se usará el valor por defecto.

```
def saludar(nombre="amigo"):
    print(f"Hola, {nombre}")
```

```
saludar()      # Output: Hola, amigo
saludar("Luis") # Output: Hola, Luis
```



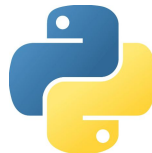


Funciones con Parámetros Arbitrarios

A veces uno sabes de antemano cuántos parámetros necesitarás. Puedes usar `*args` para pasar un número arbitrario de argumentos posicionales y `**kwargs` para pasar un número arbitrario de argumentos nombrados.

```
def sumar_todos(*args):  
    return sum(args)
```

```
print(sumar_todos(1, 2, 3))    # Output: 6  
print(sumar_todos(4, 5, 6, 7, 8)) # Output: 30
```

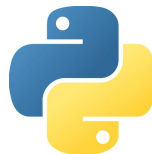




Funciones con Parámetros Arbitrarios

Un ejemplo de uso de `**kwargs`

```
def mostrar_info(**kwargs):  
    for clave, valor in kwargs.items():  
        print(f"{clave}: {valor}")  
  
mostrar_info(nombre="Ana", edad=30, ciudad="Madrid")  
# Output:  
# nombre: Ana  
# edad: 30  
# ciudad: Madrid
```

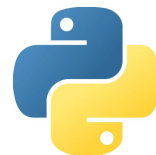




Retorno de múltiples valores

Las funciones permiten devolver múltiples valores

```
def obtener_coordenadas():  
    x = 10  
    y = 20  
    return x, y  
  
# Llamar a la función y desempaquetar los valores retornados  
coord_x, coord_y = obtener_coordenadas()  
  
print(f"Coordenada X: {coord_x}")  
print(f"Coordenada Y: {coord_y}")
```





Ejercicio integrador

Desarrolla un programa en Python que gestione la información de estudiantes y sus notas en diferentes asignaturas. El programa debe cumplir con los siguientes requisitos:

Definir Funciones:

agregar_estudiante(estudiantes, nombre, edad, ciudad): Añade un nuevo estudiante al diccionario estudiantes. Cada estudiante debe tener un identificador único (ID), nombre, edad, ciudad y un diccionario vacío para las notas.

agregar_nota(estudiantes, id_estudiante, asignatura, nota): Añade una nota para una asignatura específica a un estudiante.

obtener_promedio(estudiantes, id_estudiante): Calcula y devuelve el promedio de las notas de un estudiante.

mostrar_informacion(estudiantes): Muestra toda la información de todos los estudiantes, incluyendo el promedio de notas.

Estructura de Datos:

Utiliza un diccionario para almacenar la información de los estudiantes, donde la clave sea el ID del estudiante y el valor sea otro diccionario con la información del estudiante y sus notas.

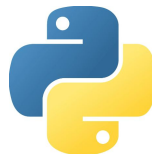
La información de cada estudiante debe ser almacenada en una tupla (nombre, edad, ciudad).

Interacción con el Usuario:

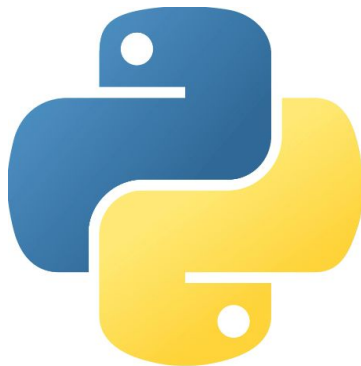
Permite al usuario agregar nuevos estudiantes.

Permite al usuario agregar notas para los estudiantes.

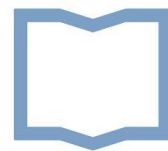
Permite al usuario obtener y mostrar la información de todos los estudiantes, incluyendo sus promedios de notas.



Próxima unidad: Clases y librerías.

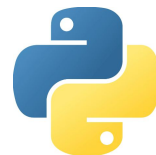
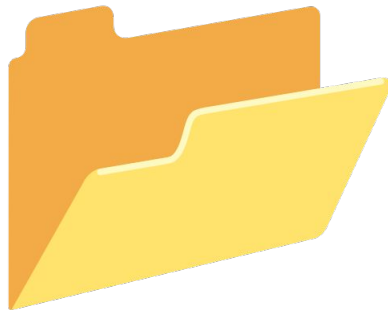


Laboratorio de computación II



Universidad
Nacional
de San Martín

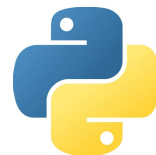
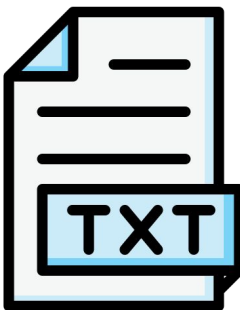
Unidad 22- Archivos en Python y Excepciones.





¿Qué es un archivo?

Un archivo es una colección de datos almacenados en un medio de almacenamiento (como un disco duro). Estos datos pueden ser de distintos tipos: texto, binarios, imágenes, videos, entre otros. En Python, existen funciones y métodos que nos permiten interactuar con archivos, manipulando su contenido de diversas maneras.





Abrir un archivo

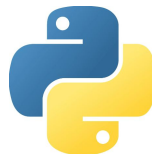
Para trabajar con archivos en Python, primero tenemos que abrir el archivo usando la función `open()`. Al abrir un archivo, Python crea una conexión entre el programa y el archivo en el disco.

```
archivo = open(nombre_archivo, modo)
```

nombre_archivo: es el nombre del archivo que queremos abrir (incluyendo su ruta si no está en el mismo directorio).

modo: es una cadena de texto que especifica cómo queremos abrir el archivo. Los modos más comunes son:

- 'r': lectura (modo por defecto).
- 'w': escritura, sobrescribiendo el archivo si ya existe o creándolo si no.
- 'a': anexar, para agregar datos al final del archivo sin borrar su contenido.
- 'b': modo binario, que se usa en conjunto con otros modos (por ejemplo, 'rb' para lectura en modo binario).



Ejemplo



Universidad
Nacional
de San Martín



```
archivo = open("mi_archivo.txt", "r") # Abre el archivo en modo lectura
```

Nota: Cuando trabajamos con archivos en Python, es importante cerrarlos al terminar para liberar recursos del sistema. Esto se hace con `archivo.close()`.



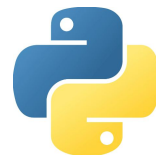


Leer archivos

Para leer el contenido de un archivo, existen tres métodos principales:

- `read()`: Lee todo el contenido del archivo y lo devuelve como una cadena.
- `readline()`: Lee una línea del archivo.
- `readlines()`: Lee todas las líneas y devuelve una lista de cadenas, cada una representando una línea del archivo.

```
archivo = open("mi_archivo.txt", "r")
contenido = archivo.read()      # Lee todo el contenido
línea = archivo.readline()      # Lee una sola línea
líneas = archivo.readlines()    # Lee todas las líneas en una lista
archivo.close()                 # Cierra el archivo
```





Leer archivos

Supongamos que tenemos un archivo datos.txt con el siguiente contenido:

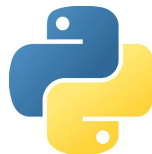


```
Python es genial.  
La programación es divertida.
```

Para leer cada línea de este archivo:



```
archivo = open("datos.txt", "r")  
for linea in archivo:  
    print(linea.strip()) # strip() elimina los saltos de línea  
archivo.close()
```





Escribir en archivos

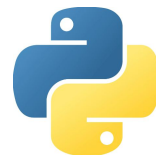
Al escribir en archivos, tenemos dos opciones principales:

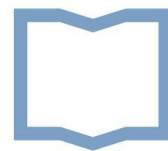
- Modo 'w': Escribe sobre el archivo. Si el archivo ya existe, borra el contenido.
- Modo 'a': Escribe al final del archivo, sin borrar el contenido existente.

Para escribir en un archivo, se utiliza el método `write()`.

```
archivo = open("mi_archivo.txt", "w") # Abre el archivo en modo escritura
archivo.write("Hola, mundo!\n")      # Escribe una línea
archivo.write("Python es divertido.\n")
archivo.close()
```

Importante: En modo 'w', si el archivo no existe, Python lo crea automáticamente.

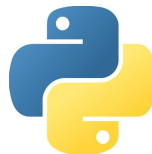


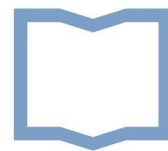


Usando el bloque with

El bloque with es la forma recomendada de trabajar con archivos en Python, ya que asegura que el archivo se cierre automáticamente, incluso si ocurre un error. Esto se conoce como context manager.

```
with open("mi_archivo.txt", "r") as archivo:  
    contenido = archivo.read()  
    print(contenido)  
# No es necesario llamar a archivo.close()
```



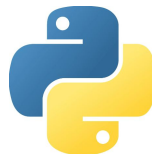


Modos de apertura combinados

Además de los modos básicos, podemos combinarlos con 'b' para trabajar con archivos binarios.

- 'rb': Lectura en modo binario.
- 'wb': Escritura en modo binario.
- 'ab': Anexar en modo binario.

```
with open("imagen.jpg", "rb") as archivo:  
    contenido = archivo.read()  
    print(type(contenido)) # tipo bytes
```





Trabajando con archivos JSON

Los archivos JSON son comunes para almacenar datos estructurados. Python ofrece el módulo json para leer y escribir en formato JSON.

```
import json

datos = {
    "nombre": "Juan",
    "edad": 25,
    "ciudad": "Madrid"
}

with open("datos.json", "w") as archivo:
    json.dump(datos, archivo) # Escribe datos en formato JSON
```



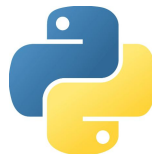


Trabajando con archivos JSON

Para poder leer un archivo json lo haremos de la siguiente manera.

```
import json

with open("datos.json", "r") as archivo:
    datos = json.load(archivo) # Carga el contenido como un diccionario
print(datos)
```





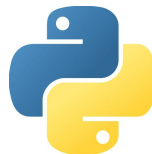
Trabajando con archivos CSV

Los archivos CSV son un formato de texto estructurado que se usa mucho en hojas de cálculo. Python incluye el módulo csv para trabajar con archivos CSV.

```
import csv

datos = [
    ["Nombre", "Edad", "Ciudad"],
    ["Juan", 25, "Madrid"],
    ["Ana", 28, "Barcelona"]
]

with open("datos.csv", "w", newline="") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerows(datos) # Escribe múltiples filas
```



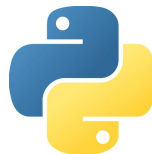


Trabajando con archivos CSV

Para leer los archivos csv podemos hacerlo de la siguiente manera:

```
import csv

with open("datos.csv", "r") as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)
```





Archivos y directorios

Para trabajar con archivos y directorios, Python incluye el módulo `os` y `os.path`.

Obtener la ruta del directorio actual:

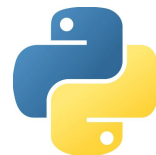
```
import os
ruta_actual = os.getcwd()
print(ruta_actual)
```

Verificar si un archivo existe:

```
import os
if os.path.exists("mi_archivo.txt"):
    print("El archivo existe.")
```

Listar archivos en un directorio:

```
import os
archivos = os.listdir(".")
print(archivos)
```





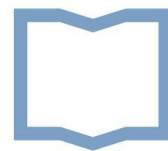
Excepciones

Una excepción es un evento que ocurre durante la ejecución de un programa y que interrumpe el flujo normal de las instrucciones. Python tiene varios tipos de excepciones ya integradas, como:

- **ValueError**: ocurre cuando se recibe un valor de tipo incorrecto.
- **TypeError**: ocurre cuando se usa un tipo de dato inapropiado.
- **FileNotFoundError**: ocurre cuando se intenta abrir un archivo que no existe.

Cuando surge una excepción, el programa se detiene. Sin embargo, podemos "manejar" esa excepción para evitar que el programa falle.



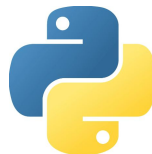


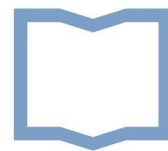
Sintaxis Básica try y except

Para capturar y manejar errores en Python, usamos la estructura try-except. Básicamente, le decimos a Python: “Intenta hacer algo, y si algo sale mal, haz otra cosa”.

```
try:
    numero = int(input("Ingresa un número: "))
    print(f"El número ingresado es {numero}")
except ValueError:
    print("Eso no es un número válido.")
```

- try contiene el código que puede fallar.
- except contiene el código que se ejecuta si ocurre el error. En este caso, si el usuario ingresa un valor que no se puede convertir a entero, ValueError se “atrapa” y se ejecuta el mensaje de error.



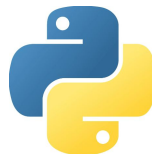


Capturar Múltiples Excepciones

A veces, diferentes tipos de errores pueden surgir en un mismo bloque de código. Podemos usar varios `except` para cada tipo de error, lo cual nos permite manejar cada caso de forma específica.

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
    numero = int(contenido)
except FileNotFoundError:
    print("El archivo no existe.")
except ValueError:
    print("El contenido del archivo no es un número.")
```

- Si el archivo `datos.txt` no existe, se captura `FileNotFoundError`.
- Si el contenido no es un número, se captura `ValueError`.





Else y Finally

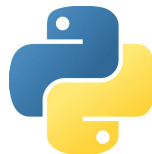
else: se usa para definir código que se ejecutará solo si no se produce ninguna excepción en el bloque try.

finally: se ejecuta siempre, independientemente de si ocurre o no una excepción. Es útil para tareas de limpieza, como cerrar archivos o liberar recursos.

```
try:
    archivo = open("datos.txt", "r")
    contenido = archivo.read()
except FileNotFoundError:
    print("No se pudo abrir el archivo.")
else:
    print("Archivo leído correctamente.")
    print(contenido)
finally:
    print("Ejecución finalizada.")
```

Si datos.txt existe, se lee y se imprime su contenido.

El mensaje "Ejecución finalizada" se muestra siempre, ocurra o no un error.

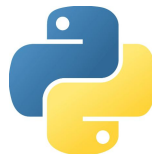




Lanzar Excepciones raise

La instrucción raise permite lanzar una excepción manualmente. Esto es útil cuando queremos validar condiciones específicas en nuestro código.

```
def dividir(a, b):  
    if b == 0:  
        raise ZeroDivisionError("No se puede dividir entre cero.")  
    return a / b  
  
try:  
    resultado = dividir(10, 0)  
except ZeroDivisionError as e:  
    print(e)
```





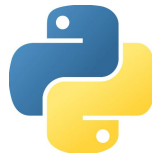
Excepciones Personalizadas

Python permite crear nuestras propias excepciones para situaciones específicas en nuestro programa. Esto es útil si queremos distinguir entre errores propios del negocio o dominio del programa y los errores generales del sistema.

```
class EdadInvalidaError(Exception):
    """Excepción lanzada cuando se ingresa una edad inválida"""
    pass

def verificar_edad(edad):
    if edad < 0:
        raise EdadInvalidaError("La edad no puede ser negativa.")
    print("Edad válida.")

try:
    verificar_edad(-5)
except EdadInvalidaError as e:
    print(e)
```



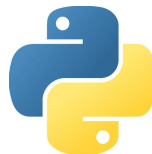


Ejemplo Completo

Este ejemplo es un flujo completo de manejo de errores en Python:

- Intenta abrir y procesar el archivo.
- Captura errores específicos para FileNotFoundError y ValueError.
- Cierra el archivo en el bloque finally para asegurarse de que el recurso se libere.

```
def procesar_archivo(ruta):  
    try:  
        archivo = open(ruta, "r")  
        contenido = archivo.read()  
        numero = int(contenido.strip())  
    except FileNotFoundError:  
        print("Error: No se encontró el archivo.")  
    except ValueError:  
        print("Error: El contenido del archivo no es un número válido.")  
    else:  
        print(f"El número en el archivo es {numero}")  
    finally:  
        archivo.close()  
        print("Archivo cerrado.")  
  
procesar_archivo("datos.txt")
```



Para terminar...

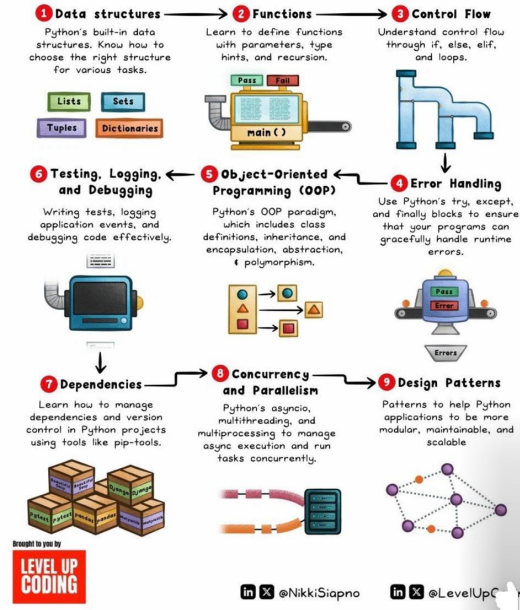


Universidad
Nacional
de San Martín

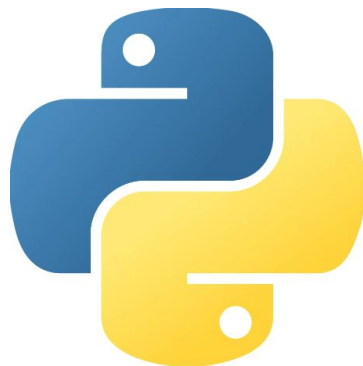


Python Roadmap

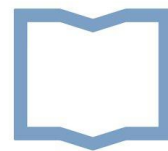
by levelupcoding.com



Próxima unidad: APIs

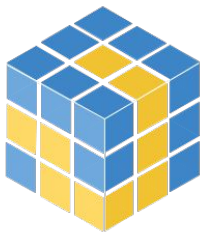


Laboratorio de computación II



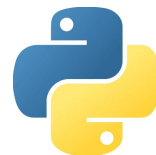
Universidad
Nacional
de San Martín

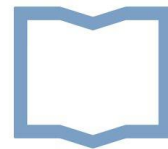
Unidad 18 - Numpy y Matplotlib.



NumPy

matplotlib





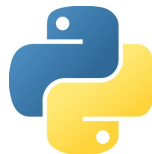
Motivación

Las listas en Python son flexibles, pero lentas para operaciones matemáticas grandes.

Necesitamos trabajar con grandes volúmenes de datos de forma rápida.

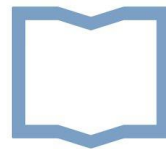
```
# Usando listas  
lista = [i for i in range(10000000)]  
suma = [x+5 for x in lista]
```

```
# Usando NumPy  
import numpy as np  
arr = np.arange(10000000)  
suma = arr + 5
```





NumPy



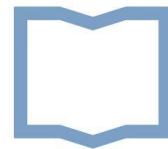
Universidad
Nacional
de San Martín

NumPy (Numerical Python) es una biblioteca de Python utilizada para realizar operaciones matemáticas y estadísticas con grandes conjuntos de datos.

- Librería fundamental para computación científica.
- Maneja estructuras llamadas ndarrays (n-dimensional arrays).
- Soporta operaciones vectorizadas (más eficientes que bucles).

```
import numpy as np  
a = np.array([1, 2, 3])  
print(a * 2)  # [2 4 6]
```



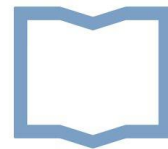


Creación de Arrays

Muy útiles para inicializar datos.

```
np.array([1, 2, 3])  
np.zeros((2, 3))    # Matriz 2x3 llena de ceros  
np.ones((3, 3))     # Matriz 3x3 de unos  
np.arange(0, 10, 2) # [0, 2, 4, 6, 8]  
np.linspace(0, 1, 5) # [0. 0.25 0.5 0.75 1.]
```





Propiedades de los Arrays

A diferencia de las listas, los arrays son homogéneos (todos los elementos del mismo tipo).

```
arr = np.array([[1,2,3],[4,5,6]])  
print(arr.shape) # (2,3)  
print(arr.ndim) # 2 dimensiones  
print(arr.dtype) # tipo de dato
```





Indexado y Slicing

Funciona parecido a las listas, pero con más potencia en varias dimensiones.

```
arr = np.array([10,20,30,40,50])  
print(arr[1]) # 20  
print(arr[1:4]) # [20 30 40]  
print(arr[::-2]) # [10 30 50]
```

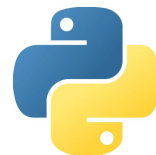


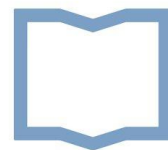


Operaciones Matemáticas

- Operaciones elemento a elemento: + - * /
- Broadcasting: NumPy ajusta automáticamente dimensiones.

```
a = np.array([1,2,3])  
b = np.array([10])  
print(a + b) # [11 12 13]
```





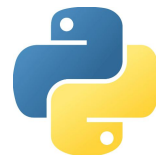
Funciones NumPy útiles

Estadísticas:

```
arr = np.random.randint(1, 100, 10)  
print(np.mean(arr), np.std(arr), np.min(arr), np.max(arr))
```

Aleatorios:

```
np.random.normal(0, 1, 1000) # distribución normal  
np.random.rand(3, 3)        # números aleatorios en  
matriz 3x3 de 0 a 1
```





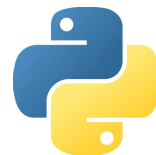
Matplotlib es una biblioteca de Python para crear gráficos estáticos, animados e interactivos. Fue creada por John D. Hunter y es ampliamente utilizada en la comunidad científica y de análisis de datos debido a su flexibilidad y potencia.

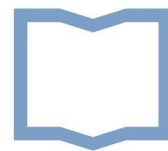
Características Principales

- Librería para visualización científica.
- Permite crear gráficos 2D y 3D.

Importación:

```
import matplotlib.pyplot as plt
```

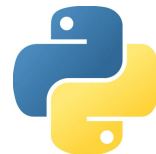
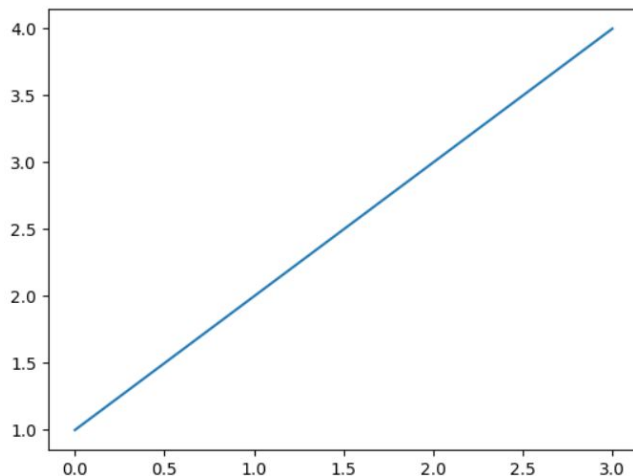




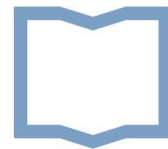
Primer gráfico

Muestra una curva simple con valores en el eje Y.

```
import matplotlib.pyplot as plt  
plt.plot([1,2,3,4])  
plt.show()
```



Matplotlib



Universidad
Nacional
de San Martín

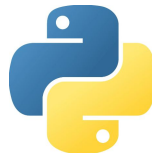
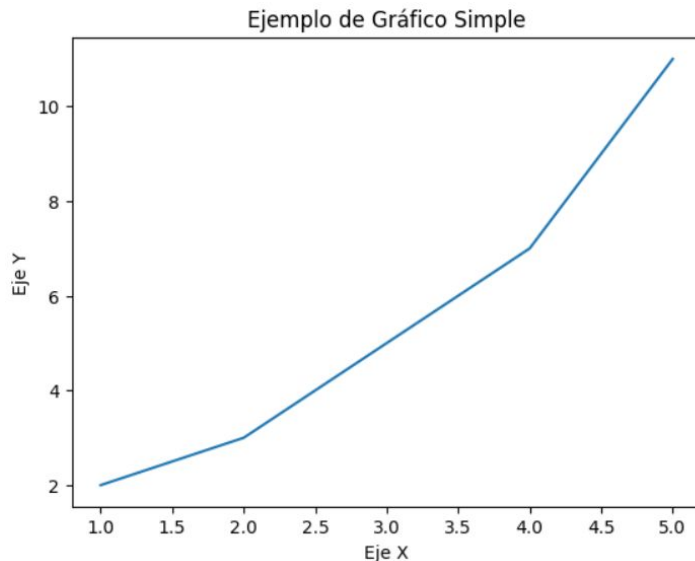
Creación de un Gráfico Simple

```
# Datos
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# Crear el gráfico
plt.plot(x, y)

# Añadir título y etiquetas
plt.title("Ejemplo de Gráfico Simple")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")

# Mostrar el gráfico
plt.show()
```





Matplotlib

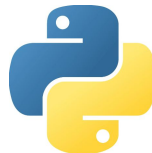
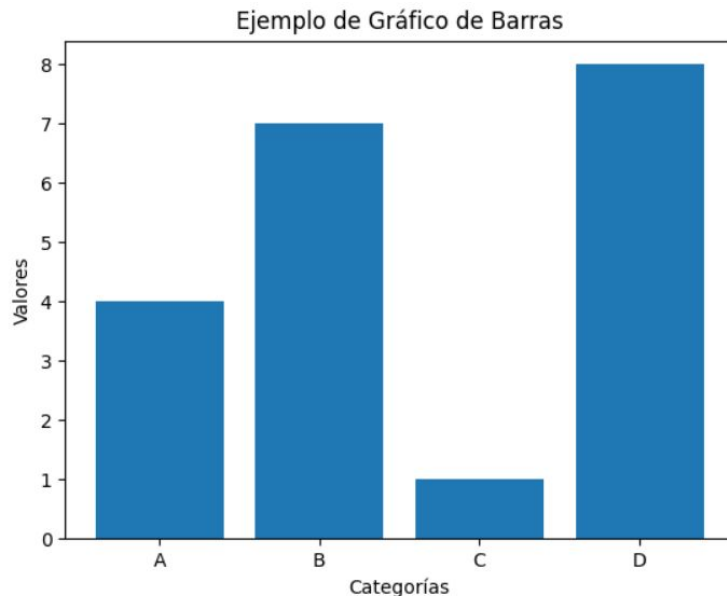
Creación de Gráfico de Barras

```
# Datos
categories = ['A', 'B', 'C', 'D']
values = [4, 7, 1, 8]

# Crear el gráfico de barras
plt.bar(categories, values)

# Añadir título y etiquetas
plt.title("Ejemplo de Gráfico de Barras")
plt.xlabel("Categorías")
plt.ylabel("Valores")

# Mostrar el gráfico
plt.show()
```





Matplotlib

Creación de Gráfico de Histograma

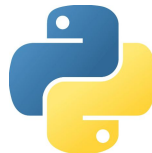
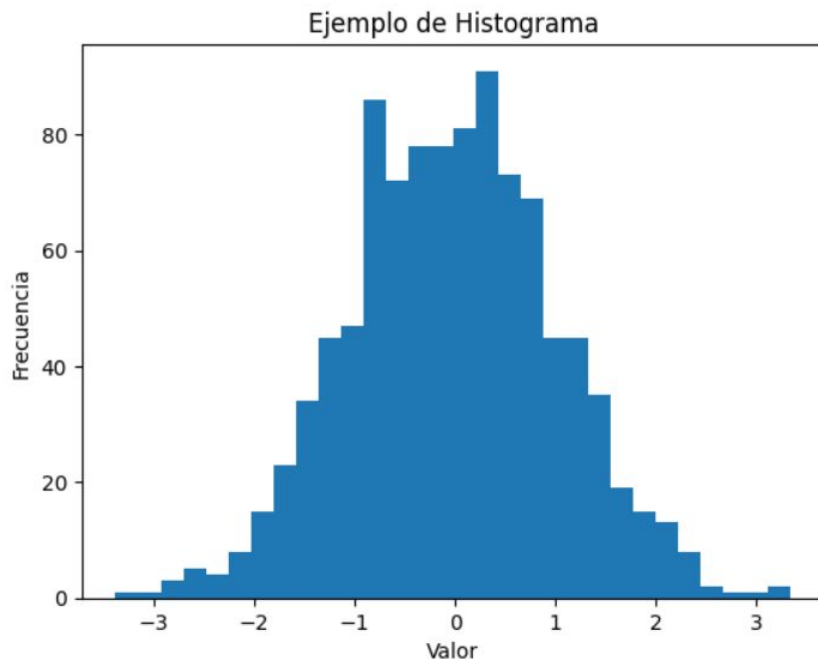
```
import numpy as np

# Datos
data = np.random.randn(1000)

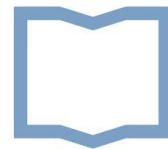
# Crear el histograma
plt.hist(data, bins=30)

# Añadir título y etiquetas
plt.title("Ejemplo de Histograma")
plt.xlabel("Valor")
plt.ylabel("Frecuencia")

# Mostrar el gráfico
plt.show()
```



Matplotlib



Universidad
Nacional
de San Martín

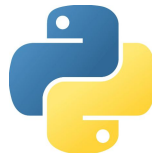
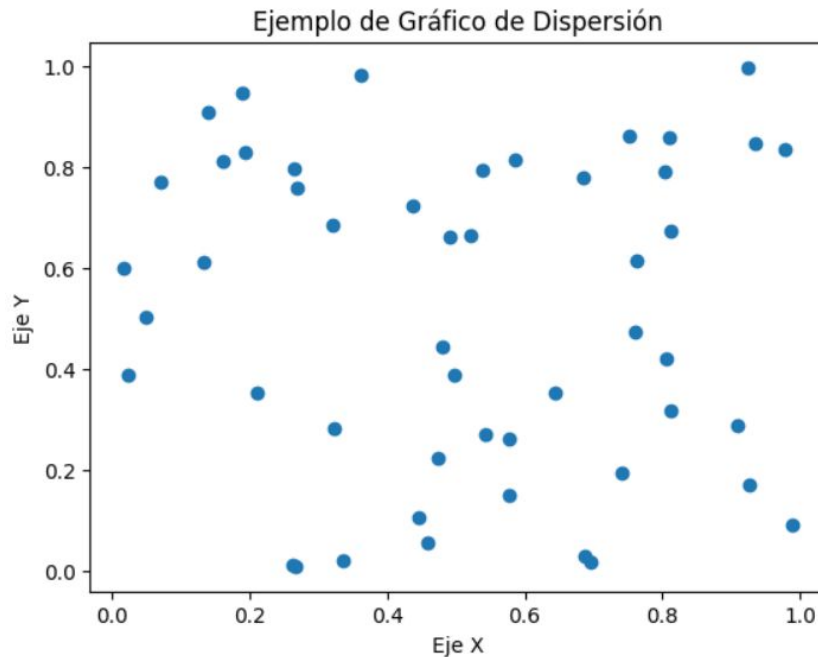
Creación de Gráfico de Dispersión

```
# Datos
x = np.random.rand(50)
y = np.random.rand(50)

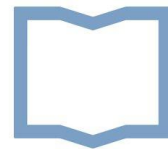
# Crear el gráfico de dispersión
plt.scatter(x, y)

# Añadir título y etiquetas
plt.title("Ejemplo de Gráfico de Dispersión")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")

# Mostrar el gráfico
plt.show()
```



Matplotlib



Universidad
Nacional
de San Martín

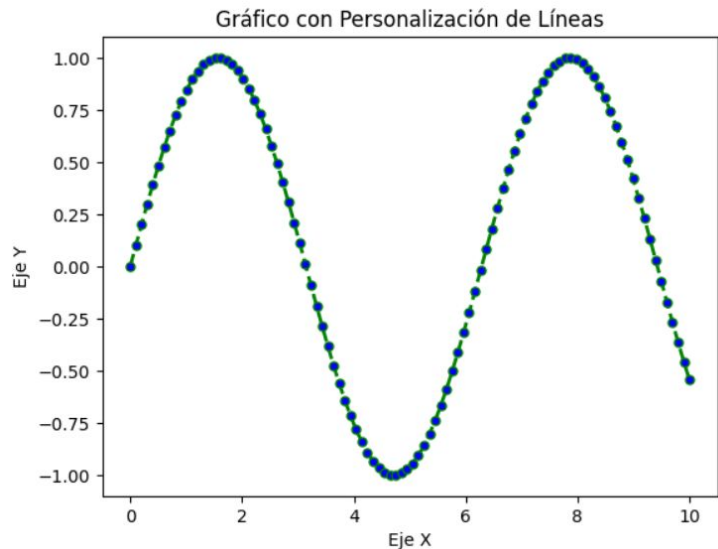
Personalización Avanzada

```
# Datos
x = np.linspace(0, 10, 100)
y = np.sin(x)

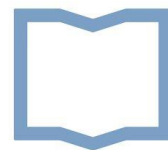
# Crear el gráfico con personalización
plt.plot(x, y, color='green', linestyle='--', linewidth=2,
marker='o', markerfacecolor='blue', markersize=5)

# Añadir título y etiquetas
plt.title("Gráfico con Personalización de Líneas")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")

# Mostrar el gráfico
plt.show()
```



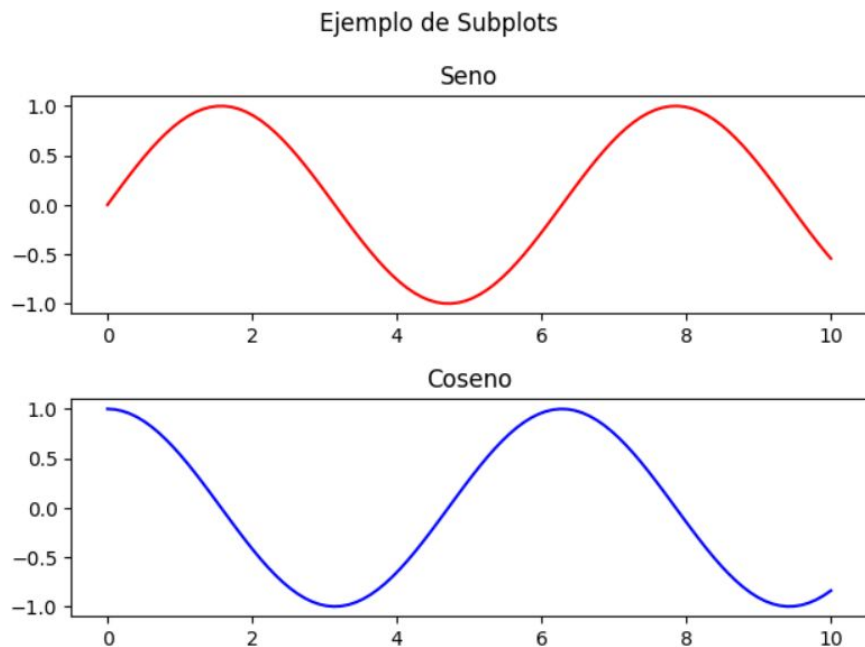
Matplotlib

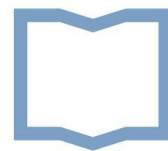


Universidad
Nacional
de San Martín

Personalización Avanzada

```
# Datos
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
# Crear los subplots
fig, (ax1, ax2) = plt.subplots(2, 1)
# Primer subplot
ax1.plot(x, y1, 'r')
ax1.set_title('Seno')
# Segundo subplot
ax2.plot(x, y2, 'b')
ax2.set_title('Coseno')
# Añadir título general y ajustar el layout
fig.suptitle('Ejemplo de Subplots')
plt.tight_layout()
# Mostrar el gráfico
plt.show()
```





Ejercicio integrador

Crear un vector de tiempo t entre 0 y 2 segundos, con 1000 muestras equiespaciadas (`np.linspace`).

Generar una señal compuesta por:

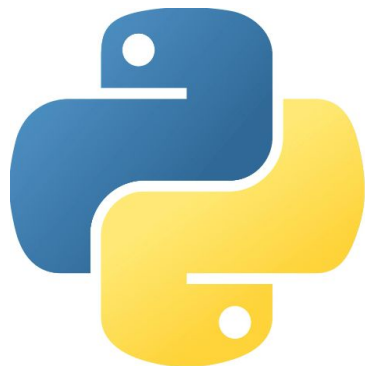
$$y(t) = \sin(2\pi \cdot 5t) + 0.5 \cdot \sin(2\pi \cdot 10t) + \text{ruido}$$

El ruido debe generarse con `np.random.normal(0, 0.3, len(t))`.

Graficar dicha señal agregando leyenda, título y ejes.



Gracias por haber cursado Laboratorio de computación II





Universidad
Nacional
de San Martín

Laboratorio de computación II

Unidad X- Limpieza de datos con Pandas

panda





Pandas

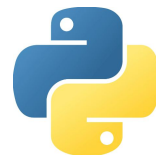
Pandas es una librería de Python diseñada para la manipulación y análisis de datos. Proporciona estructuras eficientes (Series y DataFrames) que permiten trabajar con datos tabulares y series temporales de forma sencilla, rápida y flexible.

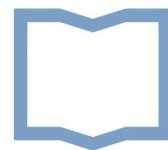
Instalación:

```
pip install pandas
```

Importación:

```
import pandas as pd
```





Series

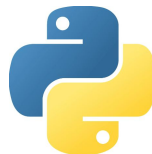
Son estructuras unidimensionales con índices y valores como los diccionarios.
Es equivalente a una columna en una hoja de calculo o tabla de bases de datos.

Inicialización de una Serie a partir de una lista con valores:

```
serie = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
```

Si no indicamos el índice, por defecto sería 0,1,2

Las Series son útiles para manejar datos de una sola característica, como una lista de temperaturas, precios de acciones o nombres de clientes. Permiten realizar operaciones eficientes sobre estos datos, como filtrado, alineación de datos basada en índices y manejo de valores faltantes.



Series

Hay varias maneras de inicializar una Serie, como por ejemplo, listas, diccionarios o valores escalares

Inicialización de una Serie a partir de una lista:

```
lista = [10,20,30]
serie = pd.Series(lista)
print(serie)
```

```
0    10
1    20
2    30
dtype: int64
```

Inicialización de una Serie a partir de un diccionario:
Toma las claves del diccionario como índice

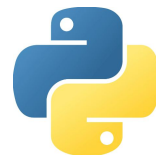
```
diccionario = {'auto': 'rojo', 'moto': 'azul'}
serie = pd.Series(diccionario)
print(serie)
```

```
auto    rojo
moto    azul
dtype: object
```

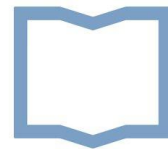
A partir de un valor escalar, va a rellenar con el valor por cada índice que tenga

```
serie = pd.Series(5, index=[0,1,2,3])
print(serie)
```

```
0    5
1    5
2    5
3    5
dtype: int64
```



Dataframes



Universidad
Nacional
de San Martín

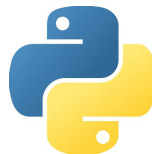
Un DataFrame es una estructura de datos bidimensional que ofrece pandas para trabajar con datos en forma tabular, como si fuese una hoja de cálculo o una tabla de base de datos. Cada columna puede tener un tipo de dato distinto (números, texto, booleanos, fechas, etc.), y cada fila representa un registro.

Hay diversas formas de como instanciar un dataframe.

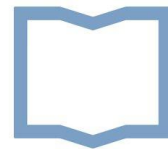
Por ejemplo desde un diccionario, donde este toma las claves del mismo como columnas y sus valores como filas

```
data = {  
    "Nombre": ["Ana", "Luis", "Juan"],  
    "Edad": [23, 35, 19],  
    "Ciudad": ["Buenos Aires", "Córdoba", "Rosario"]  
}  
  
df = pd.DataFrame(data)  
print(df)
```

	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario



Dataframes



Universidad
Nacional
de San Martín

Lista de diccionarios donde cada diccionario es una fila del dataframe

```
data = [  
    {"Nombre": "Ana", "Edad": 23, "Ciudad": "Buenos Aires"},  
    {"Nombre": "Luis", "Edad": 35, "Ciudad": "Córdoba"},  
    {"Nombre": "Juan", "Edad": 19, "Ciudad": "Rosario"}  
]  
  
df = pd.DataFrame(data)  
print(df)
```

```
↗
```

	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario

```
[64] data = (  
    ("Ana", 23, "Buenos Aires"),  
    ("Luis", 35, "Córdoba"),  
    ("Juan", 19, "Rosario")  
)  
  
df = pd.DataFrame(data, columns=["Nombre", "Edad", "Ciudad"])  
print(df)
```

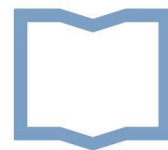
```
↗
```

	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario

Una lista de listas o tuplas, y debemos indicar las columnas que tendrá.



Dataframes



Universidad
Nacional
de San Martín

Otra forma mas común de utilizar pandas es importar un Excel o csv y pasarlo a un dataframe

Con la función `read_Excel` o `read_csv` podemos crear un dataframe a partir de esos archivos.

```
[69] df = pd.read_excel("/content/datos_ejemplo.xlsx")
```



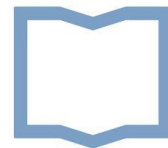
df



	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario
3	Marta	28	Mendoza



Acceso y Selección de Datos: indexación con corchetes



Universidad
Nacional
de San Martín

```
df['Nombre']
```

Nombre

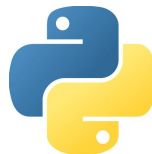
0 Ana

1 Luis

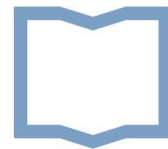
2 Juan

Podemos acceder a una columna de nuestro dataframe de esta manera. Y nos retornará una Serie de nuestro dataframe

Consultar de esta manera se puede hacer pero está deprecado(genera warnings en nuestro código), porque lo que nos devuelve es una copia de nuestro dataframe por ende no podremos modificar nuestro dataframe mediante esta forma



Acceso y Selección de Datos: loc-iloc



Universidad
Nacional
de San Martín

Pandas ofrece métodos para seleccionar filas y columnas específicas en DataFrames, estos métodos crean una vista de nuestro dataframe los que hace que nos permita modificar nuestro dataframe

Loc: el método loc nos permite acceder a un elemento por su etiqueta

```
[▶] serie.loc['a']
```

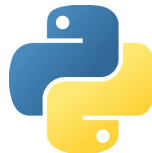
Loc: el método loc nos permite acceder a un elemento por su índice

```
[▶] serie.iloc[0]
```

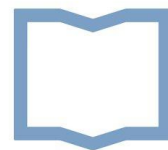
Tanto el método iloc y el método loc nos permiten seleccionar [fila, columna].

```
df.loc[0, "Nombre"]
```

```
'Ana'
```



Acceso y Selección de Datos: Slicing



Universidad
Nacional
de San Martín

El slicing en pandas nos permite seleccionar un rango de filas o columnas de manera sencilla usando el operador :

```
df.loc[0:2]
```

	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario

```
[35] df.loc[0:2, 'Edad']
```

	Edad
0	23
1	35

- La parte antes de los dos puntos indica desde qué posición queremos empezar
- La parte después de los dos puntos indica hasta qué posición queremos llegar
- Si dejamos alguno de los lados vacío, pandas interpreta que queremos todas las filas o columnas hasta el inicio o hasta el final, según corresponda.

```
[36] df.iloc[0:2]
```

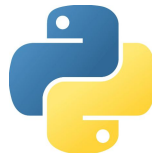
	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba

```
df.loc[0:2]
```

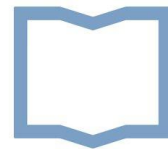
	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario

Una diferencia clave entre el slicing con iloc y con loc es que:

- En iloc, el valor límite no se incluye (exclusivo), como en las listas de Python.
- En loc, el valor límite sí se incluye (inclusivo), ya que se basa en las etiquetas del índice o columnas.



Filtrado de datos



Universidad
Nacional
de San Martín

El filtrado es clave para aislar subconjuntos de datos que cumplen con ciertas condiciones. En Pandas, esto se logra mediante el uso de expresiones booleanas.

Filtrado por valor

```
df.loc[df['Edad'] == 23]
```

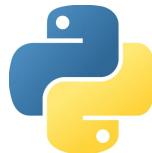
	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires

Filtrado Múltiple

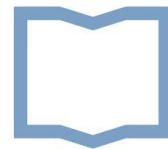
```
df.loc[(df['Edad'] >= 30) | (df['Nombre'] == 'Juan')]
```

	Nombre	Edad	Ciudad
1	Luis	35	Córdoba
2	Juan	19	Rosario

Combinando condiciones con operadores lógicos (& para AND, | para OR), podemos crear filtros complejos.



Modificar Filas y Columnas



Universidad
Nacional
de San Martín

```
df.loc[0, 'Nombre']
```

'Ana'

```
df.loc[0, 'Nombre'] = 'Annie'  
print(df)  
print(df)
```

	Nombre	Edad	Ciudad
0	Annie	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario

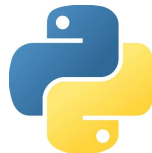
Para modificar valores específicos, puedes usar `.loc` o `.iloc`, de manera similar a cómo los usarías para la selección, pero asignando un nuevo valor.

Para modificar nombres de columnas utilizamos el método `rename` donde le pasamos un diccionario con las columnas a cambiar y su nuevo valor (`inplace=True` modifica el dataframe original)

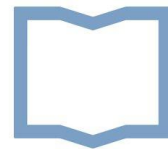
```
[96] df.rename(columns={'Edad': 'Años'}, inplace=True)
```

df

	Nombre	Años	Ciudad
0	Annie	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario



Eliminar Filas y Columnas



Universidad
Nacional
de San Martín

```
df.drop(index = 0,inplace=True)
```

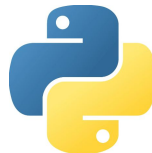
df

	Nombre	Años	Ciudad
1	Luis	35	Córdoba
2	Juan	19	Rosario

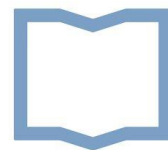
```
df.drop(columns = 'Ciudad')
```

	Nombre	Años
1	Luis	35
2	Juan	19

El método drop sirve para eliminar las filas y columnas. Además de pasar el parámetro index para filas y columns para columnas podemos utilizar axis = 0 (filas) y axis = 1 (columnas)



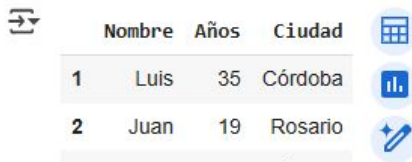
Agregar Filas y Columnas



Universidad
Nacional
de San Martín

```
newDf= pd.concat([df,df2])
```

```
newDf|
```



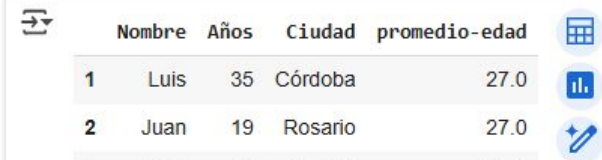
	Nombre	Años	Ciudad
1	Luis	35	Córdoba
2	Juan	19	Rosario
1	Luis	35	Córdoba
2	Juan	19	Rosario

El método concat concatena dataframes. Con elementos. Dataframes, Series, diccionarios, etc

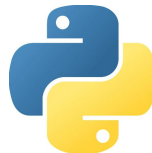
Podemos crear una nueva columna de la siguiente forma, es común utilizarlo para hacer cálculos entre columnas.

```
[116] newDf['promedio-edad'] = df.loc[:, 'Años'].mean()
```

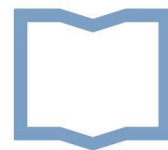
```
newDf
```



	Nombre	Años	Ciudad	promedio-edad
1	Luis	35	Córdoba	27.0
2	Juan	19	Rosario	27.0
1	Luis	35	Córdoba	27.0
2	Juan	19	Rosario	27.0



Manejo de Nulos



Universidad
Nacional
de San Martín

En nuestro set de datos podremos encontrar valores Nulos y en muchos casos vamos a querer manejarlos.

df

	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario
3	Juan	19	NaN

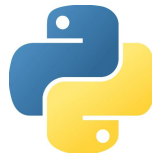
Pasos siguientes: [Generar código con df](#)

df.dropna()

	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario

El método dropna nos elimina los registros con valores nulos de nuestro dataframe

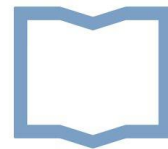
- `df.dropna(axis=1)` para eliminar columnas con valores nulos
- `df.dropna(axis=0)` para eliminar filas con valores nulos
- `df.dropna(subset=[Nombre])` para eliminar columnas específicas con valores nulos
- `df.dropna(how='all')` elimina solo si toda la fila o con `axis=1` toda la columna son



Manejo de Nulos

.fillna()

Este método se encarga de rellenar los valores nulos con un valor fijo



Universidad
Nacional
de San Martín

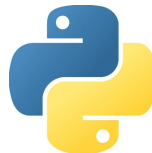
df

	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario
3	Juan	19	NaN

Pasos siguientes: [Generar código con df](#)

df.fillna(5)

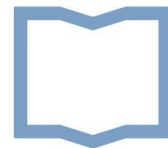
	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario



Eliminar duplicados

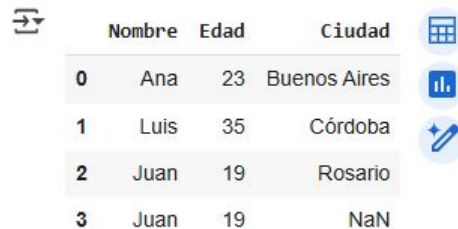
`.drop_duplicates()`

Con este método podemos eliminar los registros duplicados de nuestro dataset.



Universidad
Nacional
de San Martín

[13] df



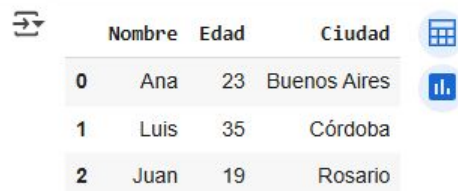
	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario
3	Juan	19	NaN

Pasos siguientes:

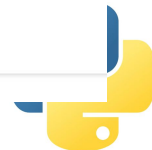
[Generar código con df](#)

[Ver gráficos recomendados](#)

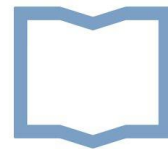
 `df.drop_duplicates(subset=['Nombre'], keep='first')`



	Nombre	Edad	Ciudad
0	Ana	23	Buenos Aires
1	Luis	35	Córdoba
2	Juan	19	Rosario



Tipo de datos



Universidad
Nacional
de San Martín

.astype() y pd.to_datetime()

El método `astype()` convierte una columna de un tipo de dato hacia otro

```
df['Edad'] = df['Edad'].astype(float)
```

```
[29] df
```

	Nombre	Edad	Ciudad
0	Ana	23.0	Buenos Aires
1	Luis	35.0	Córdoba
2	Juan	19.0	Rosario

El método `to_datetime` convierte nuestra columna al formato fecha,

```
[35] df['Nacimiento'] = df["Nacimiento"] = pd.to_datetime(df["Nacimiento"], errors="coerce")
```

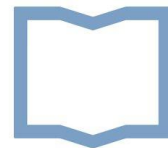
```
df
```

	Nombre	Edad	Ciudad	Nacimiento
0	Ana	23	Buenos Aires	2000-02-11
1	Luis	35	Córdoba	2000-02-11
2	Juan	19	Rosario	2000-02-11
3	Juan	19	NaN	2000-02-11

```
df['Nacimiento'] = df["Nacimiento"] = pd.to_datetime(df["Nacimiento"], format="%d/%m/%Y", errors="coerce")
```

Agrupación

.groupby()



Universidad
Nacional
de San Martín

El método `groupby()` divide nuestro dataframe según los valores de las columnas

Con las columnas agrupadas podemos aplicar una función de agregación.



	Categoria	Producto	Ventas	Cantidad
0	A	P1	100	10
1	A	P2	150	15
2	B	P1	200	20
3	B	P2	120	12
4	B	P3	300	30
5	C	P1	250	25
6	C	P2	180	18



```
df.groupby("Categoria").sum()
```



	Producto	Ventas	Cantidad
Categoria			
A	P1P2	250	25
B	P1P2P3	620	62
C	P1P2	430	43



Métodos

Ejemplo de métodos

```
[53] serie.values # devuelve un array de numpy con los valores de nuestra serie
```

```
↗ array([10, 20, 30])
```

```
[39] serie.count() #devuelve la cantidad de elementos no nulos de nuestra serie
```

```
↗ np.int64(3)
```

```
[40] serie.max() #devuelve el valor maximo de nuestra serie
```

```
↗ 30
```

```
[41] serie.min() #devuelve el valor minimo
```

```
↗ 10
```

```
[50] serie.iloc[0] #acceso a un elemento a un valor de la serie por posición
```

```
↗ np.int64(10)
```

```
[51] serie.loc['a'] #acceso a un elemento a un valor de la serie por etiqueta
```

```
↗ np.int64(10)
```

```
[57] serie.at['a'] #acceso rapido a un elemento a un valor de la serie por etiqueta
```

```
↗ np.int64(10)
```

```
▶ serie.iat[1] #acceso rapido a un elemento a un valor de la serie por posición
```

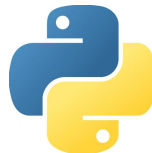
```
↗ np.int64(20)
```

```
[56] serie.dropna() #elimina los nulos de nuestra serie
```

+ Código

+ Texto

Documentación Series: <https://pandas.pydata.org/docs/reference/series.html>



Generar archivo

Con pandas podemos generar o editar un archivo csv o Excel a partir de un dataframe

De la siguiente forma:

```
df = pd.DataFrame(data)

# Guardar el DataFrame en un archivo Excel
df.to_excel('mi_archivo.xlsx', sheet_name='Datos', index=False)

print("Archivo Excel creado exitosamente.")
```

↗ Archivo Excel creado exitosamente.

Recibe como parámetro la ruta donde va a ir nuestro archivo o el que queremos sobrescribir,

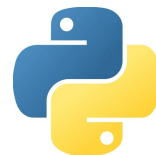
El nombre de la hoja que vamos a escribir.

El parámetro `index=False` evita que los índices del DataFrame se escriban en el archivo.

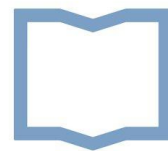
Y veremos nuestro archivo generado:



mi_archivo.xlsx



Proceso de ETL y ELT



Universidad
Nacional
de San Martín

Los procesos ETL (Extraer, Transformar, Cargar) y ELT (Extraer, Cargar, Transformar) son metodologías esenciales en la gestión de datos. Pandas es una herramienta invaluable para la fase de Transformación en ambos enfoques.

ETL

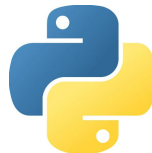
- Datos se extraen de la fuente. Se transforman (limpian, agregan, formatean) en un área de staging.
- Luego se cargan en el destino final (ej. un data warehouse).

Pandas se usa intensamente en la fase de Transformación para aplicar lógica de negocio y preparar los datos.

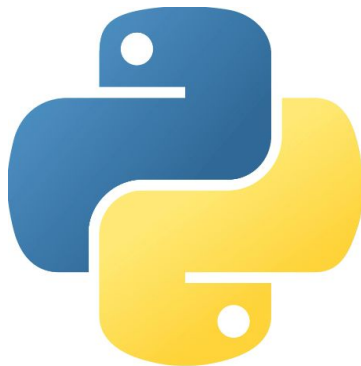
ELT

- Datos se extraen y se cargan directamente en el destino (generalmente un data lake o data warehouse moderno).
- La transformación ocurre dentro del destino, aprovechando su poder de procesamiento.

Pandas puede ser útil para transformaciones previas a la carga o para el análisis exploratorio una vez cargados los datos "en crudo".

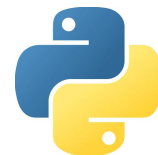


Próxima unidad: Clases y librerías.



Laboratorio de computación II

Unidad 20 - Decoradores.



Qué es un decorador ?

Un decorador en Python es una función que toma otra función como argumento, le añade cierta funcionalidad y devuelve una nueva función.



Es una forma práctica de modificar el comportamiento de una función sin cambiar su código fuente, y se utiliza frecuentemente para añadir funcionalidades, como manejo de errores, registros (logs), control de acceso, etc.

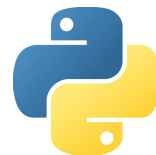




Sintaxis

La sintaxis para aplicar un decorador es el símbolo `@`, seguido del nombre de la función decoradora justo antes de la definición de la función que se va a decorar.

```
@mi_decorador
def mi_funcion():
    print("Ejecutando función")
```



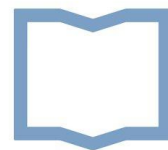


Estructura

Un decorador se define como cualquier otra función en Python, pero debe aceptar otra función como argumento y devolver una nueva función.

```
def mi_decorador(func):  
    def envoltura():  
        # Aquí puedes agregar código antes  
        print("Antes de ejecutar la función")  
        func() # Llamada a la función original  
        # Y aquí puedes agregar código después  
        print("Después de ejecutar la función")  
    return envoltura
```



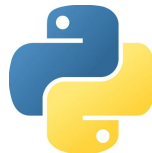


Estructura

Al aplicarse a una función, el decorador "envuelve" el comportamiento de la función original:

```
def mi_decorador(func):  
    def envoltura():  
        # Aquí puedes agregar código antes  
        print("Antes de ejecutar la función")  
        func() # Llamada a la función original  
        # Y aquí puedes agregar código después  
        print("Después de ejecutar la función")  
    return envoltura
```

```
@mi_decorador  
def saludo():  
    print("¡Hola!")  
  
saludo()
```



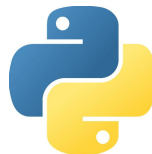


Uso básico de decoradores

El siguiente ejemplo muestra un decorador que imprime mensajes antes y después de la ejecución de una función:

```
def logger(func):  
    def wrapper():  
        print(f"Ejecutando {func.__name__}")  
        func()  
        print(f"{func.__name__} ejecutada")  
    return wrapper  
  
@logger  
def saludar():  
    print("¡Hola, Mundo!")  
  
saludar()
```

Aquí `@logger` hace que la función `saludar` se envuelva en `wrapper`, que imprime mensajes antes y después de ejecutar `saludar`.





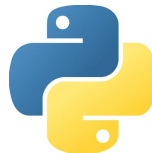
Decoradores con argumentos

Si la función que decoramos acepta argumentos, el decorador debe manejar esos argumentos. Se logra usando `*args` y `**kwargs` en la función decoradora.

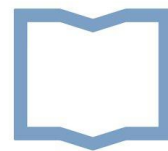
```
def logger(func):
    def wrapper(*args, **kwargs):
        print(f"Ejecutando {func.__name__} con argumentos {args} y {kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} ejecutada, resultado: {result}")
        return result
    return wrapper

@logger
def suma(a, b):
    return a + b

suma(5, 10)
```



Decoradores anidados



Universidad
Nacional
de San Martín

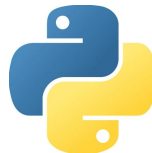
Pueden usar múltiples decoradores para una misma función aplicándolos en orden. Esto permite combinar distintas funcionalidades.

```
def decorador1(func):
    def wrapper():
        print("Ejecutando decorador 1")
        func()
        print("Decorador 1 finalizado")
    return wrapper

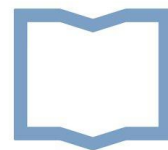
def decorador2(func):
    def wrapper():
        print("Ejecutando decorador 2")
        func()
        print("Decorador 2 finalizado")
    return wrapper

@decorador1
@decorador2
def mi_funcion():
    print("Función ejecutándose")

mi_funcion()
```



Decoradores con parámetros



Universidad
Nacional
de San Martín

Un decorador también puede recibir argumentos para ajustar su comportamiento. Para lograrlo, anidamos la función decoradora dentro de otra función que reciba los parámetros.

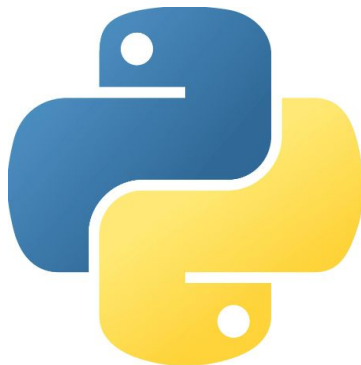
```
def repetir_veces(n):  
    def decorador(func):  
        def wrapper(*args, **kwargs):  
            for _ in range(n):  
                func(*args, **kwargs)  
        return wrapper  
    return decorador  
  
@repetir_veces(3)  
def saludar():  
    print("¡Hola!")  
  
saludar()
```



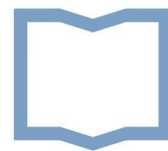


Universidad
Nacional
de San Martín

Próxima unidad: Clases, Numpy y Matplotlib.

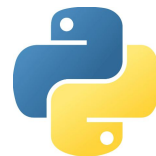


Laboratorio de computación II



Universidad
Nacional
de San Martín

Unidad 17 - Función Lambda y Comprensión .



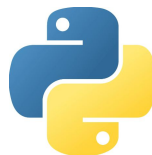


Funciones Lambda

Una función lambda es una función anónima, es decir, sin nombre.
Se define en una sola línea

lambda argumentos: expresión

Retorna el valor de la expresión automáticamente.





Ejemplo básico

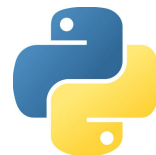
Ambas hacen lo mismo, pero la lambda es más compacta.

Función tradicional

```
def cuadrado(x):  
    return x**2
```

Función lambda equivalente

```
cuadrado = lambda x: x**2  
print(cuadrado(5)) # 25
```





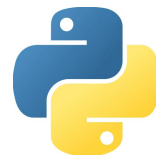
Cuando utilizarla

Usarla cuando la función:

- Es pequeña y de una sola expresión.
- No se reutiliza mucho.
- Se pasa como argumento a otra función.

No es recomendable:

- Para funciones largas o con varias líneas.
- Cuando quieras agregar comentarios o lógica compleja.





Lambda con map()

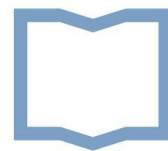
map() aplica una función a cada elemento de una lista.
su sintaxis es: `map(función, iterable)`

```
numeros = [1, 2, 3, 4, 5]
```

```
cuadrados = list(map(lambda x: x**2, numeros))
```

```
print(cuadrados) # [1, 4, 9, 16, 25]
```





Lambda con filter()

`filter()` Filtra elementos de un iterable que cumplan una condición booleana.

su sintaxis es: `filter(función_condición, iterable)`

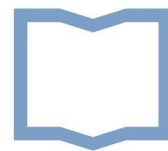
La función debe devolver `True` o `False`

```
numeros = [10, 15, 20, 25, 30]
```

```
pares = list(filter(lambda x: x % 2 == 0, numeros))
```

```
print(pares) # [10, 20, 30]
```





Lambda con sorted()

sorted() Ordena un iterable.

su sintaxis es: `sorted(iterable, key=función, reverse=False)`

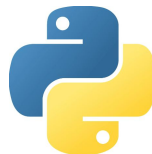
key indica el criterio de ordenación.

reverse=True → orden descendente.

```
palabras = ["python", "java", "c", "javascript"]
```

```
ordenadas = sorted(palabras, key=lambda x: len(x))
```

```
print(ordenadas) # ['c', 'java', 'python', 'javascript']
```





Lambda con sorted()

sorted() Ordena un iterable.

su sintaxis es: `sorted(iterable, key=función, reverse=False)`

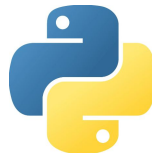
key indica el criterio de ordenación.

reverse=True → orden descendente.

```
personas = [("Ana", 20), ("Luis", 15), ("Juan", 30)]
```

```
ordenadas = sorted(personas, key=lambda x: x[1])
```

```
print(ordenadas) # [('Luis', 15), ('Ana', 20), ('Juan', 30)]
```



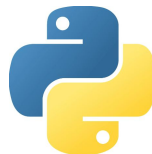


Comprensión de Listas

La comprensión de listas en Python es una sintaxis concisa y eficiente para crear listas a partir de otras iterables.

```
lista=[expresión for elemento in iterable  
condición ]
```

- **expresión** es la expresión que produce los elementos de la nueva lista.
- **elemento** es un nombre que representa cada elemento en el iterable.
- **iterable** es cualquier objeto de Python que se puede iterar, como una lista, un rango, una cadena, etc.
- **Condición** es una expresión para filtrar valores.





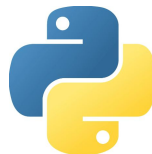
Ejemplos

Lista de cuadrados de números.

```
cuadrados = [x**2 for x in range(10)]
```

Pueden agregar una condición (if) para filtrar elementos.

```
pares = [x for x in range(10) if x % 2 == 0]
```





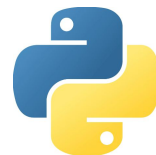
Ejemplos avanzados

Para acceder a un valor en el diccionario, usas la clave correspondiente entre corchetes [].

```
pares_de_numeros = [(x, y) for x in range(3) for y in range(3)]
```

Transformación con condicional

```
resultado = [x**2 if x % 2 == 0 else x**3 for x in  
range(10)]  
#[0, 1, 4, 27, 16, 125, 36, 343, 64, 729]
```





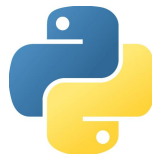
Ejemplos de uso en diferentes contextos

Trabajar con cadenas:

```
letras_mayusculas = [letra.upper() for letra in 'python']
```

Filtrar y transformar datos de una lista de diccionarios

```
personas = [{'nombre': 'Ana', 'edad': 25}, {'nombre': 'Luis', 'edad': 30}]  
mayores = [persona['nombre'] for persona in personas if persona['edad'] >  
25]  
# ['Luis']
```





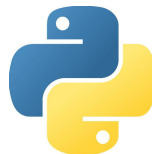
Comprensión de Diccionarios

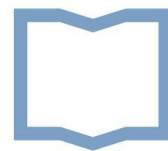
Crear un diccionario con claves y valores.

```
cuadrados = {x: x**2 for x in range(5)}  
print(cuadrados) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Filtrar solo claves pares.

```
pares = {x: x**2 for x in range(10) if x % 2 == 0}  
print(pares)  
# {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

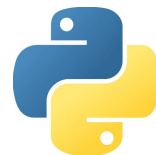


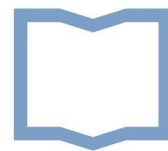


Comprensión de Diccionarios

Invertir un diccionario.

```
nombres = {"ana": 20, "juan": 25, "luis": 30}  
invertido = {v: k for k, v in nombres.items()}  
print(invertido)  
# {20: 'ana', 25: 'juan', 30: 'luis'}
```

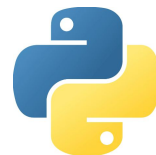




Comprensión de Conjuntos

Similar a listas, pero los resultados son únicos.

```
valores = {x**2 for x in [1, 2, 2, 3, 3, 4]}  
print(valores)  
# {16, 1, 4, 9}
```

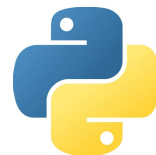




Comprensión de Conjuntos

Filtrar elementos únicos mayores que 5.

```
nums = {x for x in range(10) if x > 5}  
print(nums)  
#{6, 7, 8, 9}
```



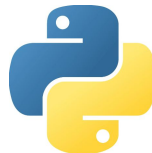


Ejercicio integrador

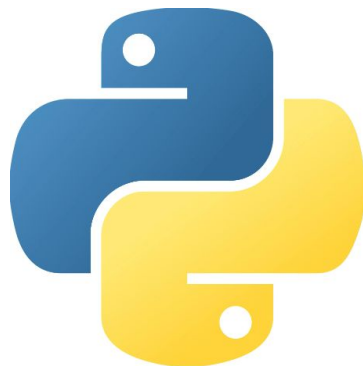
Dada la siguiente lista de personas (nombre, edad, sueldo):

```
personas = [  
    ("Ana", 20, 1200),  
    ("Luis", 15, 0),  
    ("Juan", 30, 2500),  
    ("Carla", 25, 1800),  
    ("Pedro", 17, 500),  
]
```

1. Usando filter + lambda, obtener solo las personas mayores de edad (≥ 18).
1. Usando map + lambda, quedarte solo con una lista de nombres en mayúsculas.
1. Usando sorted + lambda, ordenar las personas por sueldo (descendente).
1. Usando una comprensión de diccionario, crear un dict con {nombre: sueldo} solo de mayores de edad.
1. Usando una comprensión de conjunto, obtener todas las iniciales de los nombres de las personas.

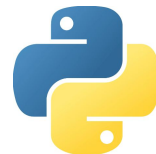
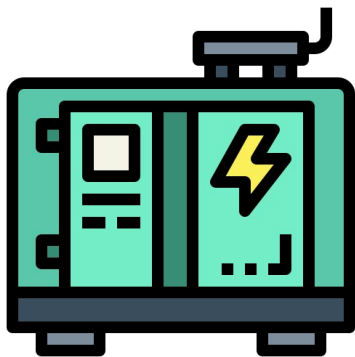


Próxima unidad: ?.



Laboratorio de computación II

Unidad 18 - Generadores y Requests .





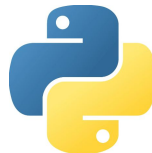
Iterador

Un iterador es un objeto que implementa dos métodos especiales:

- `__iter__()`: devuelve el propio objeto iterador.
- `__next__()`: devuelve el siguiente valor o lanza `StopIteration`.

```
lista = [1, 2, 3]
it = iter(lista) # Crea un iterador
print(next(it)) # 1
print(next(it)) # 2
print(next(it)) # 3
print(next(it)) # Error StopIteration
```

Un iterador recorre los elementos de una colección existente.



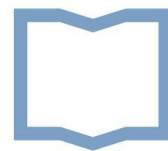


Generador

- Un generador es una forma de crear iteradores de manera sencilla.
- Se define con una función que usa `yield`.
- Guarda su estado de ejecución cada vez que produce un valor.

```
def contador(n):  
    for i in range(n):  
        yield i  
gen = contador(3)  
print(next(gen)) # 0  
print(next(gen)) # 1  
print(next(gen)) # 2
```





Funciones con generadores

Un generador es un iterador, entonces:

`next(gen)`: obtiene el siguiente valor.

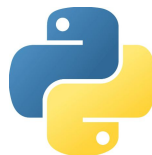
`iter(gen)`: devuelve el generador mismo.

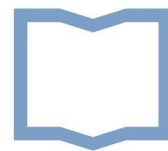
- Se puede usar en bucles:

```
for x in contador(5):  
    print(x)
```

- Se puede convertir a lista:

```
list(contador(5)) # [0, 1, 2, 3, 4]
```





Ejemplo práctico

Leer archivos grandes. No carga todo el archivo en memoria, lo lee línea a línea.

```
def leer_lineas(archivo):  
    with open(archivo) as f:  
        for linea in f:  
            yield linea.strip()  
  
for linea in leer_lineas("archivo_grande.txt"):  
    print(linea)
```



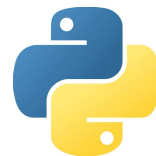


Generadores infinitos

```
def naturales():  
    n = 1  
    while True:  
        yield n  
        n += 1
```

```
gen = naturales()  
print(next(gen)) # 1  
print(next(gen)) # 2
```

Útiles para secuencias sin fin, como streams de datos.





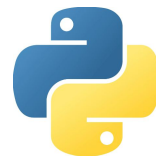
Uso de yield from

```
def subgen():  
    yield 1  
    yield 2
```

```
def gen():  
    yield from subgen()  
    yield 3
```

```
print(list(gen())) # [1, 2, 3]
```

Permite delegar en otro generador.



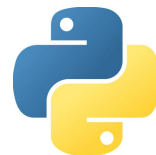


Expresiones generadoras

```
cuadrados = (x**2 for x in range(5))
```

```
print(list(cuadrados)) # [0, 1, 4, 9, 16]
```

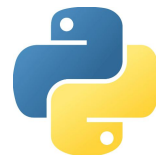
Similar a listas por comprensión, pero no guarda todo en memoria. (no es una tupla)





Conclusión de Generadores

- Son iteradores simplificados.
- Ahorran memoria y calculan valores bajo demanda.
- Ideales para procesamiento de datos grandes o secuencias infinitas.



Requests

Es una librería externa que facilita el trabajo con HTTP (el protocolo que usan los navegadores para comunicarse con servidores).

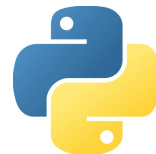
Con requests podés:

GET → traer datos de una API o página web.

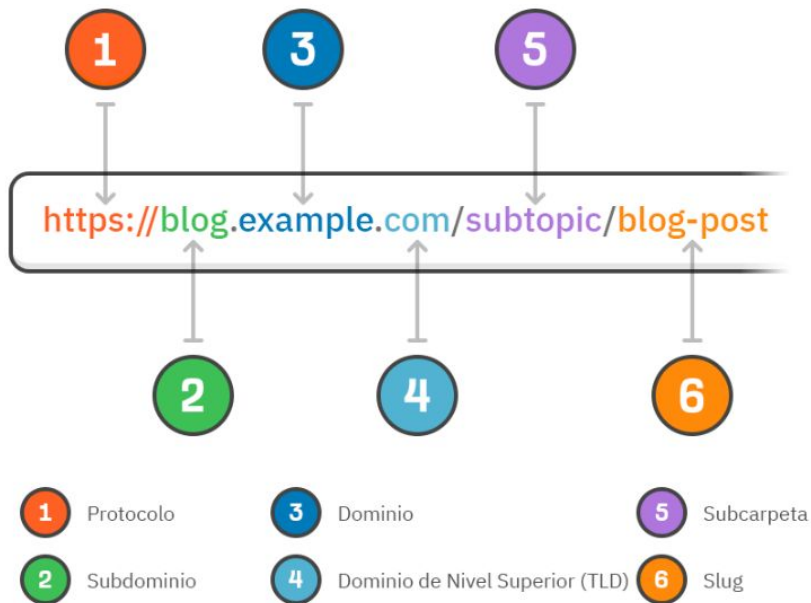
POST → enviar datos (por ejemplo, registrar un usuario).

PUT/PATCH → actualizar información.

DELETE → borrar un recurso.



Anatomía de una URL



Protocolo: Indica cómo se conecta el navegador a la web, como HTTP o HTTPS. HTTPS es preferible por su seguridad.

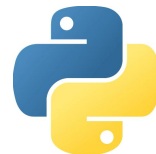
Subdominio: Parte opcional antes del dominio principal, como `www`.

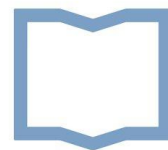
Dominio: El nombre principal de la web, como `“example”`.

TLD: La extensión del dominio, como `“.com”` o `“.es”`.

Subcarpeta: Directorios dentro de la web para organizar contenido.

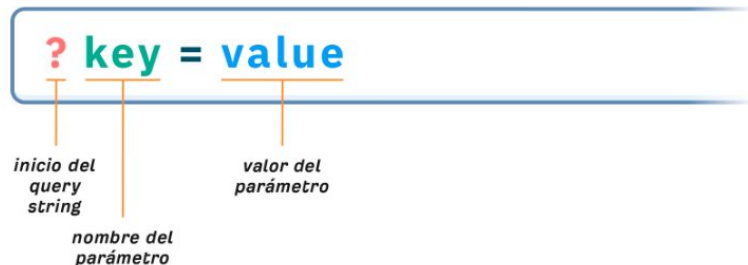
Slug: La parte específica de la URL que identifica una página concreta.





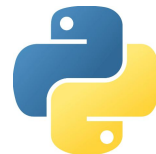
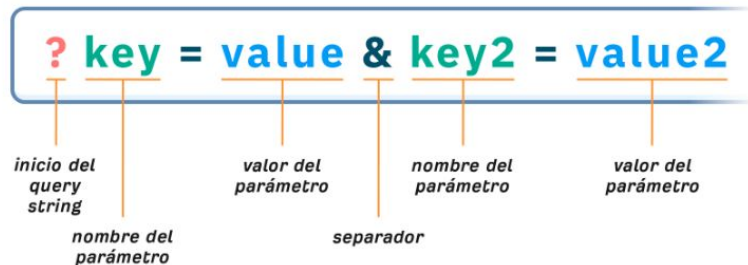
Parámetros de una URL

1 parámetro



Parámetros: Información adicional pasada a la URL, usualmente después de un signo de interrogación (?).

2 ó más parámetros



Códigos de estado HTTP



Universidad
Nacional
de San Martín

HTTP Status Codes		
Level 200	Level 400	Level 500
200: OK 201: Created 202: Accepted 203: Non-Authoritative Information 204: No content	400: Bad Request 401: Unauthorized 403: Forbidden 404: Not Found 409: Conflict	500: Internal Server Error 501: Not Implemented 502: Bad Gateway 503: Service Unavailable 504: Gateway Timeout 599: Network Timeout





Hacer una petición GET

Ejemplo: traer datos de Pikachu.

```
import requests
```

```
url = "https://pokeapi.co/api/v2/pokemon/pikachu"  
r = requests.get(url) # hacemos la petición
```

```
print(r.status_code) # 200 = éxito  
print(r.json()["name"]) # pikachu  
print(r.json()["height"]) # altura  
print(r.json()["weight"]) # peso  
# r.json() convierte la respuesta en un diccionario de Python.
```





Consultar lista de Pokémon

Ejemplo: traer lista limitada en 5 elementos.

```
url = "https://pokeapi.co/api/v2/pokemon?limit=5"  
r = requests.get(url)
```

```
data = r.json()  
for p in data["results"]:  
    print(p["name"])
```

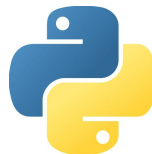




Obtener habilidades

```
url = "https://pokeapi.co/api/v2/pokemon/charmander"  
r = requests.get(url)  
data = r.json()
```

```
print("Charmander tiene las habilidades:")  
for habilidad in data["abilities"]:  
    print("-", habilidad["ability"]["name"])
```





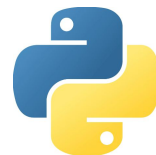
Manejo de errores

No todas las peticiones son exitosas. Por ejemplo, si pedimos un Pokémon que no existe:

```
r = requests.get("https://pokeapi.co/api/v2/pokemon/noexiste")
```

```
if r.status_code == 404:  
    print("Pokémon no encontrado")
```

Siempre es buena práctica verificar status_code.

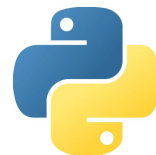




Timeout

A veces el servidor tarda mucho. Podemos evitar que nuestro programa se quede colgado con timeout:

```
try:  
    r = requests.get("https://pokeapi.co/api/v2/pokemon/ditto", timeout=2)  
    print(r.json()["name"])  
except requests.Timeout:  
    print("La petición tardó demasiado")
```

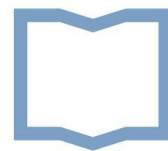




Usando generadores con Requests

```
def get_pokemon(limit=10):  
    url = f"https://pokeapi.co/api/v2/pokemon?limit={limit}"  
    r = requests.get(url)  
    for p in r.json()["results"]:  
        yield p["name"]  
  
for poke in get_pokemon(10):  
    print(poke)
```





Ejercicio integrador

Queremos construir un programa que permita consultar la PokeAPI para obtener Pokémon de un tipo específico (por ejemplo, "fire", "water", "electric") y recorrerlos uno por uno usando un generador, en lugar de traer toda la lista en memoria.

Además, cada vez que pidamos el siguiente Pokémon, se deberá mostrar:

- Nombre
- Altura
- Peso
- Lista de habilidades

Ayuda:

```
def get_pokemon_by_type(tipo):
```

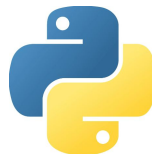
```
    """
```

```
    Generador que obtiene Pokémon de un tipo (fire, water, electric, etc.)  
    desde la PokeAPI y los entrega uno por uno.
```

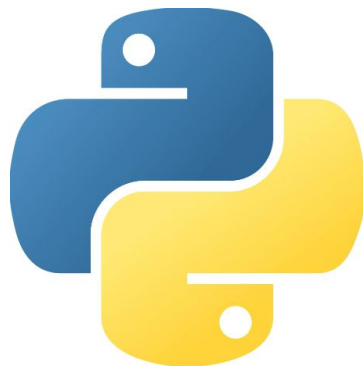
```
    """
```

```
    url = f"https://pokeapi.co/api/v2/type/{tipo}"
```

```
    r = requests.get(url)
```

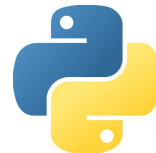
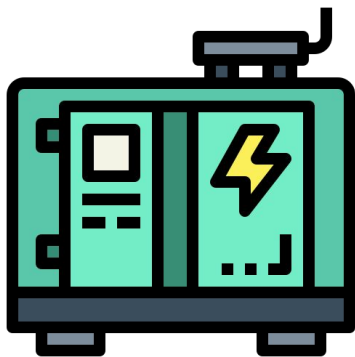


Próxima unidad: ?.

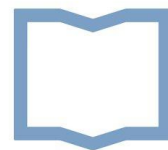


Laboratorio de computación II

Unidad 19 - Modularización



Modulos



Universidad
Nacional
de San Martín

Un modulo es un archivo.py con código que podemos importar

 funciones.py main.py

En nuestro modulo funciones
tenemos las siguientes
funciones

```
funciones.py > ...
1  def calcular_cuadrados(n):
2      return [i**2 for i in range(1, n + 1)]
3
4
5  def calcular_numeros_primos(limite):
6      primos = []
7      for num in range(2, limite + 1):
8          es_primo = True
9          for i in range(2, int(num**0.5) + 1):
10             if num % i == 0:
11                 es_primo = False
12                 break
13             if es_primo:
14                 primos.append(num)
15     return primos
```





Import

Import es la palabra reservada que utilizamos para importar código

```
import funciones  
  
print(funciones.calcular_numeros_primos(20))
```

Importamos un módulo entero

```
from funciones import calcular_numeros_primos  
  
print(calcular_numeros_primos(20))
```

```
from funciones import calcular_numeros_primos, calcular_cuadrados  
  
calcular_numeros_primos(10)  
calcular_cuadrados(5)
```

O también funciones específicas



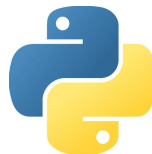


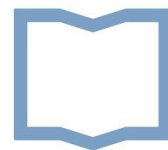
Alias (as)

```
import funciones as fn  
  
fn.calcular_numeros_primos(10)
```

A los módulos o funciones al importarlos les podemos colocar un alias con el que podemos identificarlo en nuestro código

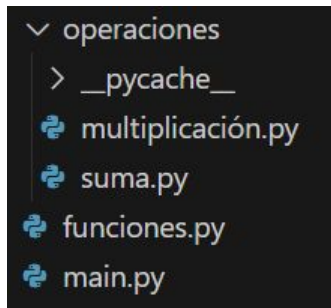
```
from funciones import calcular_cuadrados as cc, calcular_numeros_primos as cnp  
  
print(cc(5))  
print(cnp(5))
```





Paquetes

Un paquete es una carpeta con varios Módulos



```
import operaciones.suma  
  
operaciones.suma.suma([1,2,3,4,5])
```

```
from operaciones import suma  
suma.suma([1,2,3,4,5])
```

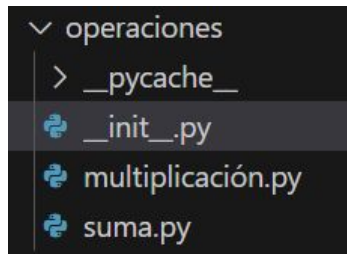
Y podemos importar los módulos del paquete de la siguiente forma





Paquetes

Podemos importar todo el paquete completo pero para eso necesitamos un archivo `_init_` en el paquete

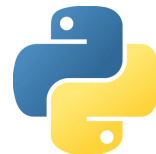


```
from .suma import *
```

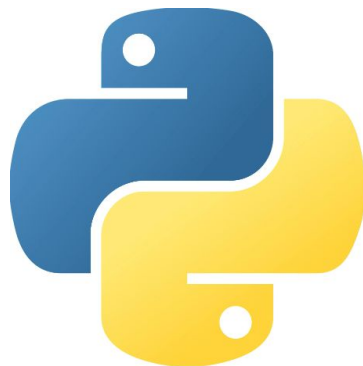
```
import operaciones

operaciones.suma([1,2,3,4,5])
operaciones.suma_de_cuadrados(3)
```

Nuestro init debe hacer la importación de nuestro modulo el “.” marca que estamos en el mismo paquete y el import * marca que vamos a importar todas las funciones de nuestro modulo

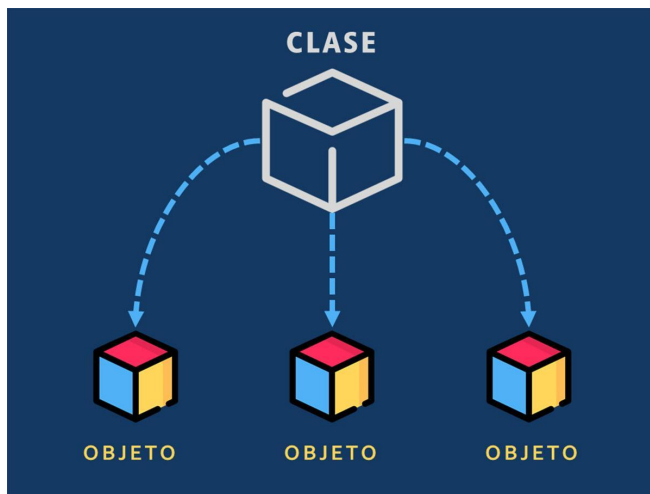


Próxima unidad: ?.



Laboratorio de computación II

Unidad 20- Clases.

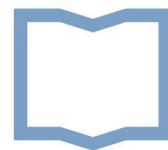




Programación Orientada a Objetos (POO)

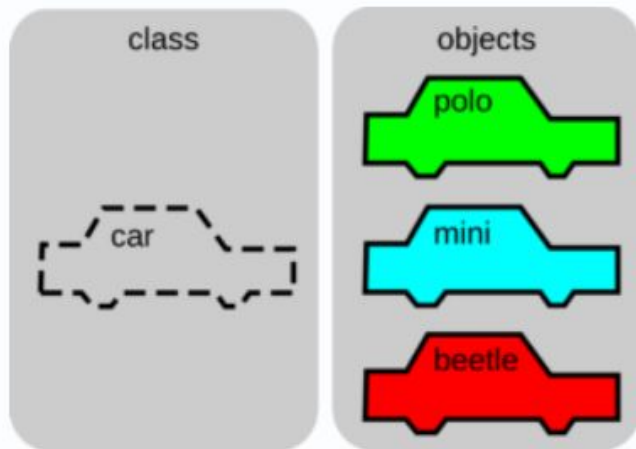
La Programación Orientada a Objetos (POO) es un paradigma de programación que se basa en el concepto de "objetos". Un objeto pretende simular entidades del mundo real.





Clases y objetos

Una clase es una plantilla o molde que define un tipo de objeto. Pueden pensar en una clase como una descripción general de un grupo de objetos que tienen propiedades y comportamientos similares.

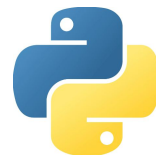




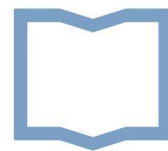
Definición de una clase

En Python, definimos una clase utilizando la palabra clave `class`, seguida del nombre de la clase (en formato PascalCase por convención).

```
class Auto:  
    pass # Clase vacía sin propiedades ni métodos
```



El Constructor



Universidad
Nacional
de San Martín

El método `__init__` es el constructor de la clase. Se llama automáticamente cuando se crea una nueva instancia de la clase y generalmente se usa para inicializar las propiedades del objeto.

```
class Auto:
    def __init__(self, marca, modelo, color):
        self.marca = marca # Propiedad de la clase
        self.modelo = modelo # Propiedad de la clase
        self.color = color # Propiedad de la clase
```

En este caso, el constructor recibe `marca`, `modelo` y `color` como parámetros y los asigna a la instancia del objeto.





Propiedades y atributos

Los atributos son variables que pertenecen a la clase o a la instancia. En este ejemplo, **marca**, **modelo** y **color** son atributos de instancia, lo que significa que cada objeto Auto tendrá su propio valor de estos atributos.

```
class Auto:
    def __init__(self, marca, modelo, color):
        self.marca = marca
        self.modelo = modelo
        self.color = color

auto1 = Auto("Toyota", "Corolla", "Rojo") # Creando una instancia de Auto
print(auto1.marca) # Salida: Toyota
print(auto1.modelo) # Salida: Corolla
print(auto1.color) # Salida: Rojo
```





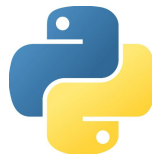
Métodos

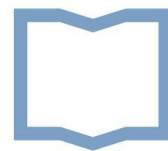
Los métodos son funciones que se definen dentro de una clase y que operan sobre los objetos de esa clase. Todos los métodos de instancia incluyen **self** como primer parámetro, que representa la instancia actual.

```
class Auto:
    def __init__(self, marca, modelo, color):
        self.marca = marca
        self.modelo = modelo
        self.color = color

    def arrancar(self):
        print(f"El {self.marca} {self.modelo} está arrancando.")

    def detener(self):
        print(f"El {self.marca} {self.modelo} está detenido.")
```

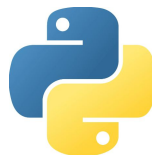


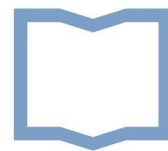


Métodos

Ahora podemos llamar a los métodos arrancar y detener desde una instancia de Auto:

```
auto1 = Auto("Toyota", "Corolla", "Rojo")
auto1.arrancar() # Salida: El Toyota Corolla está arrancando.
auto1.detener()  # Salida: El Toyota Corolla está detenido.
```





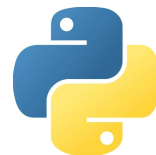
Métodos de clase y estáticos

Además de los métodos de instancia, tenemos:

- Métodos de clase: Usan `@classmethod` y reciben `cls`, que representa la clase.
- Métodos estáticos: No reciben automáticamente ni `self` ni `cls`, y se definen con `@staticmethod`.

Decoradores de Métodos

- `@classmethod`: Permite que el método reciba la clase (`cls`) como primer argumento en lugar de la instancia (`self`). Es útil para métodos que operan sobre la clase en lugar de sobre instancias individuales.
- `@staticmethod`: Declara un método que no necesita ni `self` ni `cls`. Son útiles para funciones relacionadas con la clase, pero que no requieren acceder a sus atributos o métodos.





Métodos de clase y estáticos

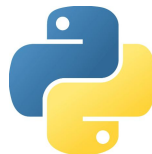
```
class Auto:
    total_autos = 0 # Variable de clase

    def __init__(self, marca, modelo, color):
        self.marca = marca
        self.modelo = modelo
        self.color = color
        Auto.total_autos += 1

    @classmethod
    def contar_autos(cls):
        return cls.total_autos

    @staticmethod
    def es_electrico(tipo_motor):
        return tipo_motor == "eléctrico"
```

```
print(Auto.contar_autos()) # Salida: 0
auto1 = Auto("Toyota", "Corolla", "Rojo")
print(Auto.contar_autos()) # Salida: 1
print(Auto.es_electrico("eléctrico")) # Salida: True
```



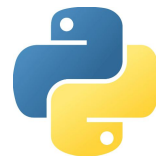


Componentes Principales de una Clase

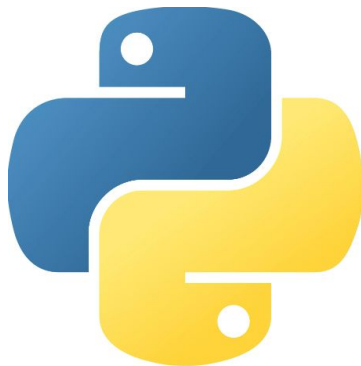
- Constructor (`__init__`): Es un método especial que se llama automáticamente cuando se crea una nueva instancia de la clase. Se usa para inicializar los atributos del objeto.
- Atributos: Son variables que pertenecen a la clase. Pueden ser de instancia (diferentes para cada objeto) o de clase (compartidos entre todas las instancias).
- Métodos: Son funciones definidas dentro de la clase que describen el comportamiento de los objetos de la clase y métodos de clase que se comparten entre todas las instancias y son propios de la clase .



Pilares de la programación orientada a objetos



Próxima unidad: Pilares de la POO.



Laboratorio de computación II

Unidad 21- Pilares de la POO.



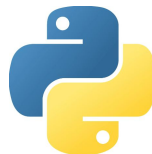


Introducción

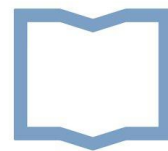
La Programación Orientada a Objetos (POO) es un paradigma que organiza el código en torno a objetos, que son instancias de clases. Los cuatro pilares de la POO son:

- Abstracción
- Encapsulamiento
- Herencia
- Polimorfismo.

Para comprender estos conceptos, utilizaremos el ejemplo de un auto, con atributos como marca, modelo, y color, y métodos como arrancar y detenerse.



Clases y Objetos



Universidad
Nacional
de San Martín

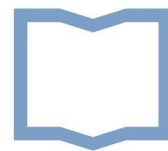
Una clase es un modelo o plantilla que define las propiedades y comportamientos que tendrán los objetos que creamos a partir de ella. Un objeto es una instancia de una clase, es decir, es algo concreto que tiene los atributos y métodos definidos en la clase.

```
class Auto:
    # Constructor de la clase
    def __init__(self, marca, modelo, color):
        # Atributos del auto
        self.marca = marca
        self.modelo = modelo
        self.color = color

    # Método para arrancar el auto
    def arrancar(self):
        return f"El {self.marca} {self.modelo} de color {self.color} ha arrancado."

    # Método para detener el auto
    def detenerse(self):
        return f"El {self.marca} {self.modelo} se ha detenido."
```



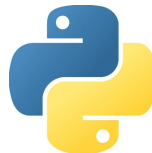


Clases y Objetos

En este ejemplo, Auto es una clase, y marca, modelo, y color son sus atributos. Los métodos arrancar y detenerse definen el comportamiento del auto.

```
mi_auto = Auto("Toyota", "Corolla", "Rojo")
print(mi_auto.arrancar()) # El Toyota Corolla de color Rojo ha arrancado.
print(mi_auto.detenerse()) # El Toyota Corolla se ha detenido.
```

Aquí mi_auto es un objeto que pertenece a la clase Auto, y puede usar los métodos arrancar y detenerse.

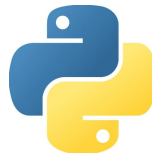




Niveles de protección

En Python, los niveles de protección de los atributos en las clases no son estrictamente controlados como en otros lenguajes como Java o C++, pero se pueden seguir ciertas convenciones para indicar el nivel de acceso deseado. Hay tres niveles principales:

- **Público:** Un atributo es público si su nombre no tiene guiones bajos al principio. Se puede acceder y modificar desde fuera de la clase.
- **Protegido:** Un atributo es protegido si su nombre comienza con un guion bajo (`_`). Se trata de una convención que indica que el atributo no debería ser accedido directamente desde fuera de la clase, pero sigue siendo accesible. Es más un aviso para los desarrolladores que una restricción real.
- **Privado:** Un atributo es considerado privado si su nombre comienza con dos guiones bajos (`__`). Python lo "renombrará" internamente para hacerlo menos accesible desde fuera de la clase, pero aún puede ser accedido de forma indirecta.

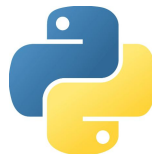




Niveles de protección

En el siguiente ejemplo se pueden observar los distintos niveles de protección.

```
class Persona:
    def __init__(self, nombre, apellido, documento):
        self.__nombre = nombre # Atributo privado
        self._apellido = apellido # Atributo protegido
        self.documento = documento # Atributo público
```



Encapsulamiento



Universidad
Nacional
de San Martín

El encapsulamiento consiste en agrupar datos y métodos dentro de una clase, y controlar el acceso a ellos. En Python, podemos controlar el acceso a los atributos utilizando diferentes niveles de protección. Esto se hace principalmente para proteger los datos y prevenir que se modifiquen desde fuera de la clase sin control.

Por convención, en Python usamos un guion bajo `_` antes del nombre del atributo para indicar que es "privado", aunque el acceso no está completamente bloqueado, sino que es más una sugerencia para que otros programadores lo respeten.



Ejemplo con encapsulamiento

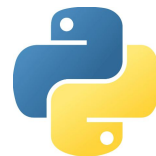
```
class Auto:
    def __init__(self, marca, modelo, color):
        self._marca = marca # Atributo "privado"
        self._modelo = modelo # Atributo "privado"
        self._color = color # Atributo "privado"

    def arrancar(self):
        return f"El {self._marca} {self._modelo} ha arrancado."

    def detenerse(self):
        return f"El {self._marca} {self._modelo} se ha detenido."

    # Método para cambiar el color del auto
    def cambiar_color(self, nuevo_color):
        self._color = nuevo_color
```

En este caso, los atributos como `self._marca`, `self._modelo`, y `self._color` están "protegidos", lo que significa que no deberían ser modificados directamente desde fuera de la clase. En su lugar, podemos utilizar métodos específicos, como `cambiar_color()`.

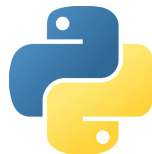




Ejemplo con encapsulamiento

Podemos utilizar métodos específicos, como `cambiar_color()`.

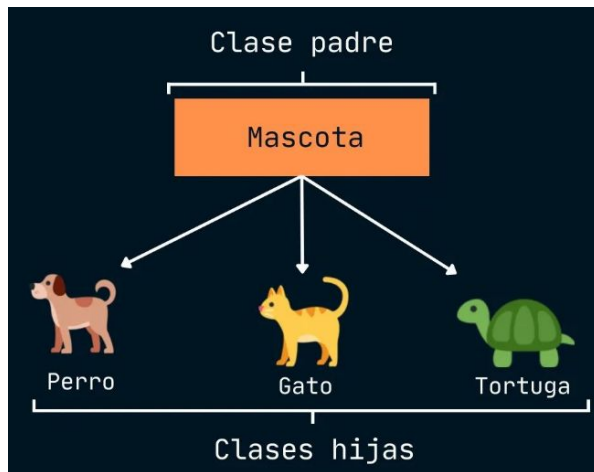
```
mi_auto = Auto("Ford", "Fiesta", "Azul")  
mi_auto.cambiar_color("Negro") # Cambiamos el color del auto a negro
```



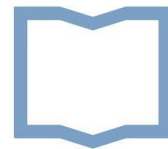


Herencia

Herencia es el mecanismo que permite a una clase tomar prestados atributos y métodos de otra clase. Esto nos permite crear nuevas clases basadas en otras ya existentes, lo que facilita la reutilización de código y la extensión de funcionalidades.



Ejemplo de Herencia



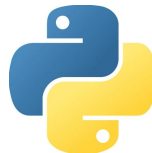
Universidad
Nacional
de San Martín

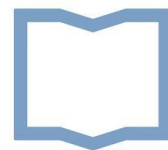
```
class AutoDeportivo(Auto):
    def __init__(self, marca, modelo, color, velocidad_maxima):
        super().__init__(marca, modelo, color) # Heredamos los atributos de Auto
        self.velocidad_maxima = velocidad_maxima # Añadimos un nuevo atributo

    def mostrar_velocidad_maxima(self):
        return f"El {self.marca} {self.modelo} tiene una velocidad máxima de {self.velocidad_maxima} km/h."
```

En este ejemplo, AutoDeportivo hereda de Auto, lo que significa que tiene todos los atributos y métodos de Auto, pero además le agregamos un nuevo atributo velocidad_maxima y un método adicional mostrar_velocidad_maxima().

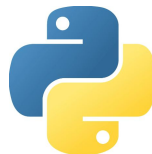
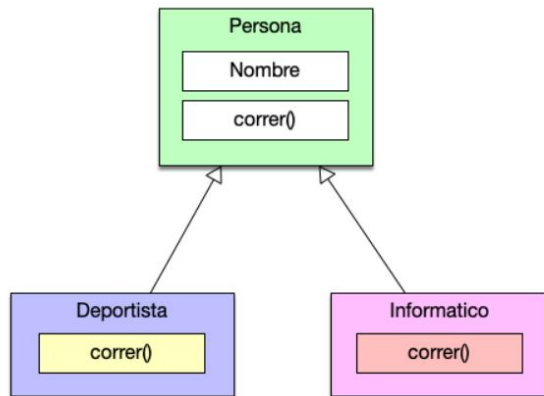
```
mi_auto_deportivo = AutoDeportivo("Ferrari", "F8", "Rojo", 340)
print(mi_auto_deportivo.arrancar()) # El Ferrari F8 ha arrancado.
print(mi_auto_deportivo.mostrar_velocidad_maxima()) # El Ferrari F8 tiene una velocidad máxima de 340 km/h.
```





Polimorfismo

El polimorfismo en POO significa "muchas formas". Es la capacidad que tienen los objetos de diferentes clases de ser tratados como si fueran de la misma clase base (común) y de responder de manera diferente a los mismos métodos. Dicho de otra manera, el mismo método o función puede comportarse de diferentes maneras según el objeto que lo invoque.





Polimorfismo

Esto es especialmente útil cuando trabajamos con herencia: las clases derivadas (hijas) pueden implementar sus propios comportamientos específicos para métodos que son definidos en una clase base (padre).

```
class Auto:
    def __init__(self, marca, modelo, color):
        self.marca= marca
        self.modelo= modelo
        self.color = color

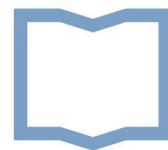
    def arrancar(self):
        return f"El {self.marca} {self.modelo} de color {self.color} ha arrancado."

class AutoElectrico(Auto):
    def __init__(self, marca, modelo, color, bateria):
        super().__init__(marca, modelo, color) # Llamamos al constructor de Auto
        self.bateria = bateria # Añadimos un atributo específico para el AutoElectrico

    def arrancar(self):
        return f"El {self.marca} {self.modelo} eléctrico ha arrancado silenciosamente."

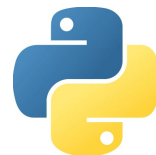
# Crear un auto eléctrico
tesla = AutoElectrico("Tesla", "Model S", "negro", "100 kWh")
print(tesla.arrancar())
```





Abstracción

La abstracción en la Programación Orientada a Objetos (POO) significa enfocarse en las propiedades y comportamientos esenciales de un objeto, mientras se ocultan los detalles complejos o innecesarios que no son relevantes para el usuario. En otras palabras, abstraemos los detalles internos para que el usuario interactúa solo con lo necesario.





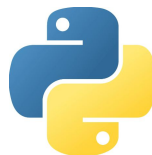
Abstracción en el código

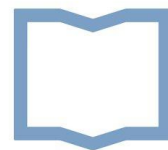
Volviendo a nuestro ejemplo del auto en Python, la abstracción es que podemos arrancar o detener el auto con métodos simples (arrancar(), detenerse()), sin tener que saber cómo esos métodos están implementados internamente.

```
class Auto:
    def __init__(self, marca, modelo, color):
        self.marca = marca
        self.modelo = modelo
        self.color = color

    # Este método "arrancar" oculta los detalles internos de cómo arranca el auto
    def arrancar(self):
        return f"El {self.marca} {self.modelo} ha arrancado."

    # Igualmente, "detenerse" es una abstracción de cómo se detiene el auto
    def detenerse(self):
        return f"El {self.marca} {self.modelo} se ha detenido."
```





¿Qué pasa cuando usamos la abstracción?

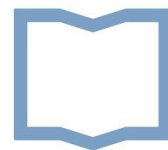
Cuando creamos un objeto de la clase Auto y llamamos a los métodos arrancar() y detenerse(), no necesitamos saber cómo estos métodos funcionan por dentro. Solo nos importa que el auto "arranca" o "se detiene":



```
mi_auto = Auto("Toyota", "Corolla", "Rojo")
print(mi_auto.arrancar()) # El Toyota Corolla ha arrancado.
print(mi_auto.detenerse()) # El Toyota Corolla se ha detenido.
```

Lo único que el usuario de la clase necesita saber es que puede llamar al método arrancar() para arrancar el auto. No le importa cómo el código realmente hace que el auto "arranque" internamente. Esto es abstracción: estamos exponiendo una funcionalidad clara y sencilla al usuario, mientras los detalles complicados se esconden en el código.





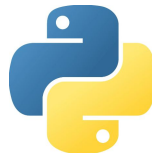
¿Qué pasa cuando usamos la abstracción?

Cuando creamos un objeto de la clase Auto y llamamos a los métodos arrancar() y detenerse(), no necesitamos saber cómo estos métodos funcionan por dentro. Solo nos importa que el auto "arranca" o "se detiene":



```
mi_auto = Auto("Toyota", "Corolla", "Rojo")
print(mi_auto.arrancar()) # El Toyota Corolla ha arrancado.
print(mi_auto.detenerse()) # El Toyota Corolla se ha detenido.
```

Lo único que el usuario de la clase necesita saber es que puede llamar al método arrancar() para arrancar el auto. No le importa cómo el código realmente hace que el auto "arranque" internamente. Esto es abstracción: estamos exponiendo una funcionalidad clara y sencilla al usuario, mientras los detalles complicados se esconden en el código.

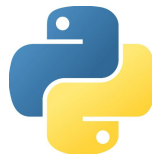


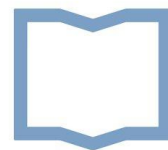


Abstracción Formal

En Python, cuando hablamos de abstracción formal en términos de implementación, nos referimos al uso de clases y métodos abstractos, y para esto se usa el módulo abc (Abstract Base Classes).

- Clases abstractas: Son clases que no pueden ser instanciadas directamente. Sirven como plantillas para otras clases. Estas clases definen métodos que deben ser implementados en las clases hijas.
- Métodos abstractos: Son métodos que se declaran en una clase abstracta pero no tienen implementación. Las subclasses están obligadas a sobrescribirlos y darles una implementación concreta.





Abstracción Formal

Imaginemos que queremos hacer una clase base llamada Vehiculo. Este vehículo debe tener métodos abstractos como arrancar() y detenerse(), pero no queremos definir cómo funcionan, ya que eso dependerá del tipo específico de vehículo (un auto, una motocicleta, etc.). Entonces, las clases que hereden de Vehiculo deben implementar esos métodos.

```
from abc import ABC, abstractmethod

# Clase abstracta
class Vehiculo(ABC):

    # Método abstracto que todas las subclases deben implementar
    @abstractmethod
    def arrancar(self):
        pass

    # Método abstracto
    @abstractmethod
    def detenerse(self):
        pass
```



Abstracción Formal

Definimos las subclases:

```
# Clase concreta que hereda de Vehiculo
class Auto(Vehiculo):
    def __init__(self, marca, modelo, color):
        self.marca = marca
        self.modelo = modelo
        self.color = color

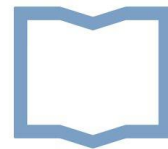
    # Implementación del método abstracto arrancar
    def arrancar(self):
        return f"El {self.marca} {self.modelo} ha arrancado."

    # Implementación del método abstracto detenerse
    def detenerse(self):
        return f"El {self.marca} {self.modelo} se ha detenido."

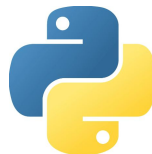
# Clase concreta que hereda de Vehiculo
class Motocicleta(Vehiculo):
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    # Implementación del método abstracto arrancar
    def arrancar(self):
        return f"La motocicleta {self.marca} {self.modelo} ha arrancado."

    # Implementación del método abstracto detenerse
    def detenerse(self):
        return f"La motocicleta {self.marca} {self.modelo} se ha detenido."
```



Universidad
Nacional
de San Martín



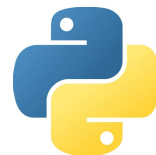


Abstracción Formal

Por último instanciamos y utilizamos las subclases

```
# Crear instancias
mi_auto = Auto("Toyota", "Corolla", "Rojo")
mi_moto = Motocicleta("Honda", "CBR")

print(mi_auto.arrancar()) # El Toyota Corolla ha arrancado.
print(mi_auto.detenerse()) # El Toyota Corolla se ha detenido.
print(mi_moto.arrancar()) # La motocicleta Honda CBR ha arrancado.
print(mi_moto.detenerse()) # La motocicleta Honda CBR se ha detenido.
```



Próxima unidad: ?

